Journal of Industrial Information Integration 42 (2024) 100731

Contents lists available at ScienceDirect

Journal of Industrial Information Integration

journal homepage: www.elsevier.com/locate/jii

Full length article

Semantic Building Information Modeling: An empirical evaluation of existing tools

Ignacio Huitzil [a,b,c,d], Miguel Molina-Solana [b,e], Juan Gómez-Romero [e], Marco Schorlemmer [c], Pere Garcia-Calvés [c], Nardine Osman [c], Josep Coll [f], Fernando Bobillo [a,g,*]

[a] University of Zaragoza, Zaragoza, Spain
[b] Imperial College London, London, United Kingdom
[c] Artificial Intelligence Research Institute (IIIA), CSIC, Bellaterra (Barcelona), Spain
[d] IKR3 Lab, University of Milano-Bicocca, Milano, Italy
[e] University of Granada, Granada, Spain
[f] Enginyeria i Project Management (EiPM), Barcelona, Spain
[g] Aragon Institute of Engineering Research (I3A), Zaragoza, Spain

ARTICLE INFO

Keywords:
Building Information Modeling
Ontologies
RDF graphs
Semantic BIM software

ABSTRACT

Semantic Building Information Modeling (BIM) consists in translating data expressed using BIM formats (namely IFC) into Semantic Web files using RDF serializations (e.g., Turtle). This enables the inference of new knowledge and constraint checking, among other advantages. While several software tools for translating BIM models into Semantic Web languages have been proposed in the literature, they differ in the features exposed.

This paper analyzes and empirically compares some of these tools (namely, IFC converters translating an input IFC model into an RDF graph), identifying their strengths and main limitations. Our methodology includes measuring computation times of common tasks (file conversion, query and inference over output files), assessing the retention of knowledge (particularly, geometric information) and examining reasoning capabilities (complexity and completeness of the resulting models). Our results show that IFCtoLBD is the best option in many cases. IFCtoRDF and IFC2LD are slower but better preserve geometric information, while KGG is faster at the expense of losing information in the translation.

1. Introduction

Building Information Modeling (BIM) is a collaborative process to manage information in a construction project [1]. BIM is based on a digital representation of a building for integrated design, modeling, planning, and operation during its whole lifecycle [2], from design to decommissioning. BIM models include many pieces of interconnected information, including a 3D geometric model of the building elements, a description of the materials used and their properties, etc. The advantages of using BIM include optimizing construction and maintenance costs, improving transparency and collaboration between different stakeholders, managing complex projects, and flexibility to adapt to changing requirements quickly. Therefore, BIM is a key element in the digitalization of the construction sector [3].

Industry Foundation Classes (IFC) is the vendor-neutral and open ISO standard for BIM data exchange [4]. The IFC specification defines a conceptual schema for BIM elements, which is represented in the data modeling languages EXPRESS (ISO 10303-11) and XSD (XML Schema Definition), and a file format, namely IFC-SPF (IFC STEP Physical Format, ISO 10303-21) and ifcXML.[1] According to [5], "IFC schema is a standardized data model that codifies, in a logical way the identity and semantics (name, machine-readable unique identifier, object type or function), the characteristics or attributes (such as material, color, and thermal properties) and relationships (including locations, connections, and ownership) of objects (like columns or slabs), abstract concepts (performance, costing), processes (installation, operations) and people (owners, designers, contractors, suppliers, etc.)".

* Corresponding author at: University of Zaragoza, Zaragoza, Spain.
E-mail addresses: ihuitzil@unizar.es (I. Huitzil), miguelmolina@ugr.es (M. Molina-Solana), jgomez@decsai.ugr.es (J. Gómez-Romero), marco@iiia.csic.es (M. Schorlemmer), pere@iiia.csic.es (P. Garcia-Calvés), nardine@iiia.csic.es (N. Osman), pcoll@eipm.es (J. Coll), fbobillo@unizar.es (F. Bobillo).

[1] There are other serialization formats: IfcZIP, JSON, HDF5, and SQLite, see https://wiki.osarch.org/index.php?title=IFC_-_Industry_Foundation_Classes for details.

https://doi.org/10.1016/j.jii.2024.100731
Received 8 September 2023; Received in revised form 25 October 2024; Accepted 3 November 2024
Available online 12 November 2024
2452-414X/© 2024 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

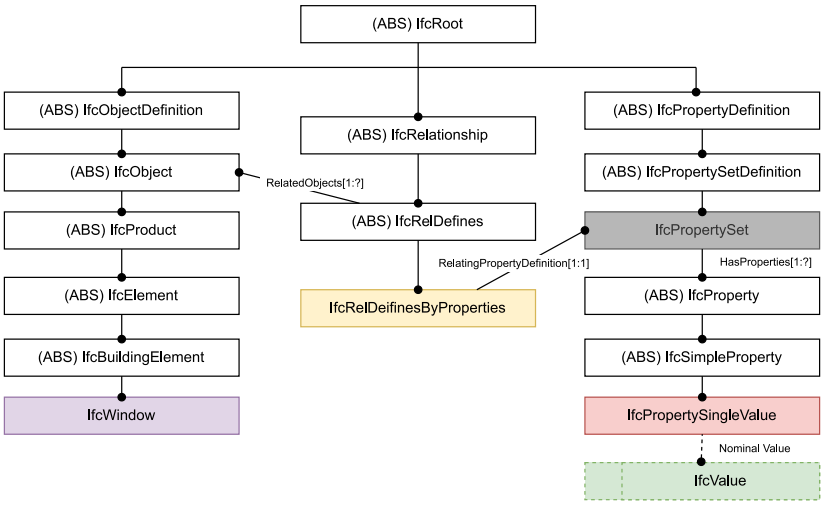


Fig. 1. IFC EXPRESS diagram to represent windows.

```
#6921=IFCWINDOW('1hOSvn6df7F8_7GcBWlRLx',#33,'M_Fixed:750mm x 2200mm:750mm x 2200mm:146885'
    ,$,'750mm x 2200mm',#6920,#6915,'146885',2.199999999999996,0.7499999999999996);
...
#6940=IFCPROPERTYSINGLEVALUE('Height',$,IFCLENGTHMEASURE(2.2),$);
...
#6943=IFCPROPERTYSINGLEVALUE('Width',$,IFCLENGTHMEASURE(0.7500000000000001),$);
...
#6993=IFCPROPERTYSET('3wXESTX8r65A35RpVNAwPp',#33,'PSet_Revit_Type_Dimensions',$,
   (#6940,#6942,#6943,#6944));
...
#38580=IFCRELDEFINESBYPROPERTIES('002GW$n7bBQR51XrlqHj8z',#33,$,$,
   (#6921,#7025,#22396,#22448),#6993);
```

Fig. 2. Extract of an IFC-SPF file.

For example, Fig. 1 shows an extract of an IFC EXPRESS schema including the entities to represent a window building element (IfcWindow) and a set of properties (IfcPropertySet), for example, properties with a nominal value (e.g. IfcPropertySinglevalue). Fig. 2 shows a textual representation of an IFC window in IFC-SPF. Colors are used to represent the same elements in the different figures. For example, the text in purple represents an IfcWindow with a unique ID 6921.

There are several versions of the IFC schema (IFC2x3_TC1, IFC4, IFC4_ADD1, IFC4_ADD2, IFC4_ADD2_TC1, IFC4x1, and IFC4x3_RC1). Nowadays, IFC2x3_TC1 is the most used version in the Architecture, Engineering, and Construction (AEC) sector, although IFC4 incorporates several extensions.[2]

There have been several attempts to evolve BIM into *semantic BIM* [6], combining BIM with Semantic Web languages including OWL (Web Ontology Language) [7,8] and RDF (Resource Description Framework) [9]. OWL is the standard language to represent ontologies, which are formal and shared specifications of the vocabulary of a domain of interest [10]. For example, ifcOWL ontology uses OWL to represent IFC elements [11]. RDF is a meta-model from which other Semantic Web languages, suc has as OWL and RDFS, are derived [9].

IFC converters are software tools that transform input IFC models into a semantic BIM model expressed as RDF triples. Developing an IFC converter tool is challenging, as they need to manage large volumes of heterogeneous data (e.g., geometric data, material types, providers, etc.) Although many IFC converters are available, they offer different features. Therefore, it is difficult for users of IFC converters to choose the most appropriate software for their needs.

Following a mixed-methods approach, this work performs an analysis of different IFC converter tools, namely IFC2LD, IFCtoRDF, IFCtoLBD, and Knowledge Graph Generator or KGG (see Section 4 for details). Some theoretical features of each tool are discussed, and the results of an empirical comparison, including real-world datasets, are reported. The results compare the tools using many criteria, such as the computation time, the preservation of elements, and the reasoning capabilities, among others.

Specifically, this paper addresses the following Research Questions regarding current available implementations:

RQ1 Can the tools convert an input IFC file quickly and robustly (i.e., it does not crash despite the input size and format)?

RQ2 What is the size of the converted files and how many ontology elements (i.e., classes, properties, individuals, etc.) are there?

RQ3 How similar are the results of the converter tool and the original model, i.e., are some parts of the input file lost on translation?

RQ4 How appropriate are the converted files to answer SPARQL queries in terms of size of the answers and execution time?

RQ5 How appropriate are the converted files to perform semantic reasoning using OWL reasoners in terms of reasoning time?

[2] See https://app.bimsupporters.com/courses/ifc/lessons/history-and-versions-of-ifc/ for a comprehensive list of differences between the IFC schema versions.

This paper is structured as follows. Section 2 overviews related work, and Section 3 provides some preliminaries on Semantic Web languages and ontologies. Then, Section 4 introduces and compares, from a theoretical point of view, the software tools that will be analyzed. Next, Section 5 describes our methodology and experimental setup. The results of our experiments are presented in Section 6 and discussed in Section 7. Section 8 summarizes the main findings and points towards directions for future work.

2. Related work

The IFC format has many advantages, but its limitations have also been pointed out. Pauwels, Zhang and Lee remarked interoperability issues among software tools used in diverse disciplines, interoperability among application domains, and the inability to discover implicit knowledge using automated reasoning [12]. Luo et al. highlighted the needs to enhance current BIM tools (to improve flexibility, standardization, and interoperability), to develop ontologies or schemas to represent metadata of construction data, and to integrate building data across the building lifecycle to support various use cases [13].

To solve such problems, there have been several attempts to evolve BIM into *semantic BIM*. For a comprehensive overview of semantic BIM, the interested reader is referred to [6]. Pauwels, Zhang and Lee stated the following motivations for semantic BIMs [12]: (1) facilitating interoperability and information exchange between heterogeneous tools, (2) linking cross-domain information to exploit synergies of related domains, and (3) equipping AEC data models with logic-based representation capacities. These authors concluded that many research gaps remain unexplored, such as combining declarative and procedural techniques and automating data integration and retrieval procedures. Some examples of applications of semantic BIM in the field of construction are constraint checking [14,15], formal verification of IFC models [16], or fuzzy logic-based flexible querying [17,18].

A fundamental ontology in the construction area is ifcOWL [11,19,20] (see Section 3.2 for a formal definition of ontologies). ifcOWL provides a semantic mapping for the IFC elements in the OWL 2 language and thus represents the IFC model. For example, it supports classes such as ifc:IfcWindow, individuals such as ifc:OAK, properties of the objects such as ifc:overallHeight, or relationships between objects of the model such as ifc:madeOf. IFC objects are usually represented as classes, except for IFC properties which can be modeled as different types of properties. This results in a very complex ontology structure not correctly handled by existing editors and reasoners.

Actually, there are different versions of the ifcOWL ontology.[3] Given an input EXPRESS/STEP file, the algorithm in [19] produces an OWL file which is an OWL serialization of the EXPRESS file and thus it follows the version of the IFC schema. For example, the ifcOWL ontology automatically generated from the EXPRESS schema IFC4 (https://standards.buildingsmart.org/IFC/DEV/IFC4/FINAL/OWL/index.html) includes IfcActuator, which is not included in the version generated from IFC2X3 (https://standards.buildingsmart.org/IFC/DEV/IFC2x3/FINAL/OWL/index.html). Furthermore, some tools define versions of the ontology using different fragments of OWL, as we will see in Section 4.

IFC makes it possible to represent many different types of geometric information. According to Fangzheng et al. the geometry of building elements is mainly represented using the IFC classes IfcShapeRepresentation, IfcBoundingBox and IfcGeometricCurverSet [21]. IfcShapeRepresentation includes 2D/3D points, curves, and surfaces, solid models, etc. IfcBoundingBox represents elements with its own local coordinate system. IfcGeometricCurveSet refers to a list of at least one or more 2D or 3D points and curves, including polylines. For more details about geometry types supported by IFC, we refer the reader to [22].

Managing geometric information in semantic BIMs has received significant attention. Since it is not efficient to store geometric data in OWL ontologies, Pauwels and Roxin proposed to simplify them to obtain a smaller size of the semantic version of the BIM model [23]. Pauwels et al. also proposed several alternatives to maintain the data, improving the RDF representation using an optimization of geometrical data in the semantic representation (commonly represented as an aggregation data type, `rdf:list`) and splitting into small ontologies specific for the user's application [24]. Ouyang et al. provided another solution based on separating the semantic (BIM) model from the geometric (BIM) data, which is implemented in the tool CDE (Common Data Environments) [25]. However, CDE does not store geometric data in a semantic format, so the semantic BIM does not have all the data in the IFC model.

In summary, IFC is a very complex structure that, when mapped as an OWL ontology, requires adaptations and simplifications. Since existing IFC converters have very different features and behavior, analyzing such differences is the purpose of this paper.

3. Preliminaries

This section revises some basic notions on the Semantic Web technologies that will be used in the rest of the paper.

3.1. RDF and OWL

RDF (Resource Description Framework) is a standard formalism for data interchange [9,26]. It uses a triple representation (s, p, o) where s is a subject (a resource), o an object (a resource or literal), and p is a predicate or property. For instance, ⟨*Window01, ifc:objectPlacement, b01*⟩ is a triple with a straight-forward meaning: "Window01 is placed at b01".

An RDF model can be expressed using many syntax notations (or serializations), such as TURTLE (or Turtle), TURTLE.GZ, N-TRIPLES, N-QUADS, RDF/XML, and JSON-LD [27]. RDF/XML is the standard format, so it is widely supported by RDF tools, and it encodes the information in XML. JSON-LD encodes the information using JSON data interchange format. None of them are friendly for a human user. Instead, TURTLE is a compact syntax which is more readable by humans. TURTLE.GZ is a variant which uses gzip compression. N-TRIPLES is an easy-to-parse line-based subset of TURTLE, but it is hard to interpret by humans. N-QUADS is similar to N-TRIPLES but it supports multiple graphs. Appendix A provides more details about the Turtle syntax, the one that will be used in the examples of this paper.

Every element of an RDF triple is identified by a Uniform Resource Identifier (URI), such as <https://standards.buildingsmart.org/IFC/DEV/IFC2x3/TC1/OWL#Window01> Sometimes, only URI fragments (omitting the prefix with the namespace, such as Window01) are presented rather than complete URIs, as in the example in the first paragraph. RDF includes some special properties. For example, `rdf:type` can be used to state

[3] http://standards.buildingsmart.org/IFC/, in "DEV" and "RELEASE" folders.

that a subject entity belongs to a class. RDF also supports a special type of nodes, *blank nodes*, representing anonymous resources. Blank nodes can be the subject or the object of an RDF triple, but not the property. RDF blank nodes are represented with the syntax `[ ]` or `_:x`.

An *RDF graph* is a set of RDF triples, where the subject and the object of a triple are nodes and the property is an oriented edge between them. Explicit triples are RDF triples that are included in an RDF graph. Inferred (or implicit) triples are not explicitly stated but can be logically deduced from other triples.

SPARQL is the standard query language for RDF [28]. It is used to retrieve triples from a data graph that satisfy a given condition. It also includes a protocol for addressing queries and getting results over HTTP.

RDFS (RDF Schema) is an extension of RDF with a specific vocabulary to represent subclasses (`rdfs:subclassOf`), domain (`rdfs:domain`) and range (`rdfs:range`) restrictions, and subproperties (`rdfs:subpropertyOf`) [29]. However, this vocabulary is insufficient for many applications in which a more expressive language (i.e., with a richer set of well-defined elements) is needed.

OWL (Web Ontology Language) is the recommendation of the World Wide Web Consortium (W3C) for such cases [8]. The current version is OWL 2 [7]. OWL is designed on top of other technologies in the Semantic Web stack [30], such as the eXtensible Markup Language (XML) as a serialization format, and RDF as a metamodel. Based on them, OWL offers an additional vocabulary and formal semantics to describe more complex scenarios. For example, it enables the representation of equivalent classes (`owl:equivalentClass`), intersection of classes (`owl:intersectionOf`) or existential restrictions (`someValuesFrom`), whose semantics cannot be represented in RDF (see Example 2 for an example). The RDF triples in an RDF graph can be represented as OWL axioms. However, in general, the semantics of OWL axioms cannot be represented using RDF.

The standard defines three sublanguages (tractable *profiles*) of OWL 2, namely OWL 2 EL, OWL 2 QL and OWL 2 RL [31]. All of them limit the expressiveness power for the sake of computational efficiency, having a polynomial-time reasoning complexity. Formally, OWL 2 is equivalent to the Description Logic $\mathcal{SROIQ}(\mathbf{D})$ [7]. Typically, the name of a Description Logic is a concatenation of capital letters denoting the constructors that it supports. For example, in $\mathcal{SROIQ}(\mathbf{D})$, $\mathcal{O}$ indicates that the language includes nominal concepts. By removing different elements of OWL, other less expressive Description Logic languages can be obtained. For instance, $\mathcal{SROIF}(\mathbf{D})$ is a sublanguage of $\mathcal{SROIQ}(\mathbf{D})$ obtained by replacing qualified cardinality restrictions ($\mathcal{Q}$) to the particular case of functional properties ($\mathcal{F}$). Differences between Description Logic can be found at [32].

3.2. Ontologies

An ontology is a formal and shared specification of the knowledge in a domain [33]. Specifically, an OWL ontology is a set of axioms represented using OWL. Advantages of ontologies include promoting knowledge sharing and reuse, being easy to read and process for machines (software applications) and humans (natural form), providing a division of the domain knowledge and the application level (which is particularly useful for intelligent systems) and making it possible to reason by discovering pieces of knowledge that logically derives from the explicitly represented ones.

The main ingredients of ontologies to model the knowledge in a domain are individuals (or instances), classes (or concepts), datatype properties, object properties, and axioms (the interested reader can find more details in [7]).

Individuals represent the entities of the domain. For instance, Window01 is an individual.

Classes are sets of individuals. For example, the IfcWindow class represents all windows in the IFC vocabulary. Classes are typically structured in a class hierarchy (subclass–superclass) or taxonomy. For example, BasicWindow is a subclass of IfcWindow.

Properties (or roles) represent binary relationships between a pair of elements in the domain. There are two types of properties: *object properties* relate a pair of individuals, and *data properties* (or datatype properties) link an individual with a data value (of a specificXSD datatype, such as integer or a string). For example, the object property objectPlacement links a window (such as Window01) and a place (such as b01). Moreover, the data property overallHeight links a particular window (such as Window01) with a number (2190) denoting its height.

Axioms are constraints or restrictions to be verified by the elements of the ontology. Axioms in an ontology can be organized using two sets [32]: a Terminological Box or TBox (with axioms about concepts and roles such as subclass axioms and concept definitions),[4] and an Assertional Box or ABox (with axioms about individuals or facts). Every ontology has a TBox, but not every ontology has an ABox. In this paper, we will consider subclass axioms and three types of ABox axioms: a concept assertion states that an individual belongs to a concept, an object property assertion states that two individuals are related via an object property, and a data property assertion relates an individual and a datatype value via a data property.

Finally, the expressivity of an OWL 2 ontology is defined by the different Description Logic constructors it uses (such as functional properties) and denoted by capital letters, as already mentioned in Section 3.1.

3.3. Semantic reasoning

Reasoning is probably the most distinguishing feature of ontologies, as it plays an important role both in the development of models (to verify their correctness) and in the deployment of real-world applications (to discover knowledge). A *reasoner* [34] is a software implementation of algorithms solving some reasoning tasks. In other words, it infers implicit knowledge from explicit knowledge in a logically consistent way.

Several semantic reasoners support OWL 2 ontologies and provide different reasoning tasks, such as ontology consistency checking (i.e., checking if the ontology can have a model satisfying all axioms) or classification (i.e., computing a concept and property hierarchy based on the subclass relationship) [32].

In this study, three semantic reasoners were considered: *HermiT* [35],[5] *TrOWL* [36],[6] and *Pellet* [37].[7] All of them can be accessed from Java applications and the OWL API [38],[8] a Java API to manage OWL ontologies. While HermiT and Pellet support full OWL 2, TrOWL only supports OWL 2 EL (for more expressive ontologies, it computes a syntactic approximation into OWL 2 EL).

[4] Sometimes, the TBox is restricted to axioms about concepts, and axioms about roles are moved to a Role Box or RBox [32].
[5] http://www.hermit-reasoner.com
[6] https://github.com/TrOWL/core
[7] https://github.com/stardog-union/pellet
[8] http://owlapi.sourceforge.net

Many RDF triple stores only support answering SPARQL queries involving the explicitly represented information. However, some graph database systems also have integrated inference engines to infer RDF triples. Virtuoso,[9] which uses its own inference module, and GraphDB,[10] which uses the Triple Reasoning and Rule Entailment Engine TRREE,[11] are examples of commercial tools, whereas Apache Jena[12] is an open source framework.

4. IFC to RDF conversion tools

This section describes the four IFC converters analyzed in this paper. They all convert IFC models in IFC-SPF (STEP) format used in the construction domain into Semantic Web documents in Turtle syntax, except IFC2LD, which supports more RDF formats.

IFC2LD. It is the short name for IFC-to-Linked Data.[13] In some versions, the tool is also called IFC2RDF[14] and IFC-to-RDF.[15] In this paper, we use the old name IFC2LD to avoid confusions with the next tool IFCtoRDF.

IFC2LD implements a multilayer conversion where each layer adds detailed information (such as property type (data or object), property ranges, keys of entity classes, or property cardinalities) about the objects and properties in the model.

As an input, this converter accepts the IFC schemas IFC2x3_TC1, IFC4, and IFC4_ADD1. The tool settings make it possible to configure OWL 2 profiles, output RDF formats (TURTLE, TURTLE.GZ, NTRIPLES, NQUADS, RDFXML_ABBREV, RDFXML_PRETTY, and JSONLD), and level of information.

IFC2LD involves two translation steps. Firstly, it computes the schema. To do so, ifcOWL ontology is firstly adapted to the specific IFC EXPRESS schema of the input file. Then, three possible subontologies of increasing expressivity (called *layers*) can be used, namely ifcOWL-Simple, ifcOWL-Standard and ifcOWL-Extended.[16] *ifcOWL-Simple* is compatible with the intersection between OWL 2 EL, OWL 2 QL, and OWL 2 RL. *ifcOWL-Standard* is an ontology in OWL 2 RL. *ifcOWL-Extended* is an ontology in OWL2 DL. In the second translation step, IFC data are represented as RDF triples with respect to the computed schema. The output is called *ifcRDF* by the authors of the tool [40] and also includes geometrical data of the BIM models.

In IFC2LD, it is also possible to submit not only the input IFC file but also an .exp file with the ifcOWL ontology already adapted to the specific IFC EXPRESS schema.[17] This is very useful because the tool sometimes fail to compute the adapted ifcOWL ontology.[18]

IFCtoRDF. This tool is called IFC-to-RDF in [41]. It translates an IFC-represented BIM model into an RDF graph. In particular, it generates a set of triples which can be seen as an ABox with respect to an ifcOWL TBox [41]. ifcOWL ontology is adapted to the specific IFC EXPRESS schema of the input file. Note that this ifcOWL ontology is different to the one used by IFC2LD, as IFCtoRDF does not define fragments of ifcOWL adapted to different OWL 2 profiles (see details in Table 6). Additionally, geometric data of building elements in IFC can be saved in the RDF output. It supports several versions of the IFC schema (IFC2x3_TC1, IFC4, IFC4_ADD1, IFC4_ADD2, IFC4_ADD2_TC1, IFC4x1, and IFC4x3_RC1). IFCtoRDF has two end-user interfaces: a Command Line Interface[19] (CLI) recommended by the buildingSMART International, and a Graphical User Interface[20] (GUI).

IFCtoLBD. It stands for IFC to Linked Building Data. IFCtoLBD uses a similar translation process as IFCtoRDF, in the sense that the output can use different schemes. Since the first versions, IFCtoLBD converts BIM models (IFC files) into RDF graphs with respect to the BOT, PRODUCT, and PROPS ontologies [41]. The Building Topology Ontology (BOT)[21] provides an ontology of classes and properties to describe the topological elements of a building [42]. PRODUCT[22] consists of two modular ontologies: BEO[23] and MEP.[24] The Building Element Ontology (BEO) contains a vocabulary to describe the common building elements as a subclass of the IfcBuildingElement IFC class. The Distribution Element Ontology (MEP) describes a taxonomy of classes for common distribution elements as a subclass of the IfcDistributionElement, such as IfcFlowTerminal. Finally, the PROPS ontology[25] is a TBox defining properties for building-related elements and is in the conceptual design phase. More recently, the tools also supports OMG[26] and OPM[27] ontologies. The Ontology for Managing Geometry (OMG) describes dimensions, spatial relationships, and the geometrical shapes of building components. The Ontology for Property Management (OPM) is an ontology for describing temporal properties during the operational phase of a building's lifecycle.

[9] http://virtuoso.openlinksw.com
[10] https://www.ontotext.com/products/graphdb/
[11] https://graphdb.ontotext.com/documentation/10.0/architecture-components.html#architecture-components-trree
[12] https://jena.apache.org
[13] We downloaded the tool from http://drumbeat.cs.hut.fi/ifc2ld, the link provided in [39], which does not currently work. In the "software" and "specification" folders, there was a link to https://github.com/Web-of-Building-Data/ifc2ld.git.
[14] Actually, https://github.com/Web-of-Building-Data/ifc2ld.git currently redirects to https://github.com/Web-of-Building-Data/Ifc2Rdf.
[15] https://github.com/Web-of-Building-Data/Ifc2Rdf/blob/master/software/distributives/export-ifc2rdf/1.5.0-SNAPSHOT/docs/IFC-2-RDF%20Converter_User%20manual_1.5.0.pdf
[16] http://drumbeat.cs.hut.fi/owl/
[17] https://technical.buildingsmart.org/standards/ifc/ifc-schema-specifications
[18] For example, this happended in the 7-dwelling building. When IFC2LD processes IfcCartesianPointList2D, it failed to convert IfcTemporaryCollectionValueWrapper into IfcLiteralValue, but it was solved by using the file https://standards.buildingsmart.org/IFC/RELEASE/IFC2x3/TC1/EXPRESS/IFC2X3_TC1.exp.
[19] https://github.com/pipauwel/IFCtoRDF
[20] http://github.com/jyrkioraskari/IFCtoRDF-Desktop
[21] http://w3c-lbd-cg.github.io/bot/bot.ttl
[22] https://github.com/pipauwel/product
[23] https://pi.pauwel.be/voc/buildingelement/index-en.html
[24] https://github.com/pipauwel/product/blob/master/mep.ttl
[25] https://lbd.arch.rwth-aachen.de/LBD/ontology.ttl
[26] https://annawagner.github.io/omg/archive/0-3/
[27] https://w3c-lbd-cg.github.io/opm/

Table 1
Functional and non-functional features of the converters tools.

Converter tool	Last version	Language	API for	OS	Interface	Origin	License	Price
IFC2LD	1.5 (13/03/2015)	Java	Java REST	Windows Unix	CLI	AU	MIT	Free
IFCtoRDF	0.4 (29/08/2020)	Java	Java	Windows	CLI	UGent	Apache V2.0	Free
	2.8.1 (28/08/2020)			Apple OS X GNU/Linux	GUI	AU Apache		
IFCtoLBD	2.43.5 (14/02/2024)	Java	Java, Python Open API Docker BIMserver plugin	Windows Apple OS X GNU/Linux	GUI	TU/e KU Leuven AU UGent RWTH Aachen	Apache V2.0	Free
KGG	0.2.0 (11/01/2024)	Java	Java	Multi-platform	Web	UCL	Unknown	Free

Actually, IFCtoLBD only uses 5 BOT classes[28] (bot:Site, bot:Building, bot:Storey, bot:Space, and bot:Element), and 3 BOT properties (bot:adjacentElement, bot:containsElement, and bot:hasSubElement).

PROPS supports 3 levels of complexity.[29] The levels range from 1 to 3 and indicate the length of a path between an object and the value of its property, and therefore the complexity of the model.[30] Level $L1$ consists of a single link (the property) that connects a subject and the property value. Level $L2$ has two links: the first one links the subject and an intermediate node, and the second one links the intermediate node and the property value. The intermediate node can have additional properties or states. Level $L3$ consists of three predicates between the object and the property value, with two internal nodes.

During the conversion process, IFCtoLBD has a first internal step using the IFCtoRDF converter to encode the ABox as a set of RDF triples. After that, the ifcOWL classes (resp. properties) are matched with the classes (resp. properties) of BOT, PRODUCT, or PROPS. Therefore, the IFC building elements are connected via some properties in a resumed ABox represented as an RDF graph (or multiple graphs according to the user settings). The IFCtoLBD desktop GUI version[31] allows the selection of the complexity levels of the PROPS ontology (Levels 1, 2, and 3) and the use of blank nodes, as well as the number of output files, as the result can be split into several RDF files. Optionally, this converter makes it possible to omit geometric data[32], reducing the size of the triples and query time. Both CLI and desktop versions support seven standard IFC schemes (e.g. from IFC2x3_TC1 until IFC4x3_RC1).

KGG. It stands for Knowledge Graph Generator.[33] Our research considered the online version.[34] The architecture of this converter is based on IFCtoLBD. KGG translates IFC data to RDF triples with respect to the ontologies BRICK, BEO, BOT, and PROPS [43]. This converter aims to retrieve and combine data from the building topology (BOT), building products (BEO, PROPS, and BRICK) and sensors (BRICK). The BRICK ontology[35] captures the building operation phases. It was created for Smart Building projects such as sensors (Internet of Things), Building Management Systems, or mechanical systems [44]. While BOT defines the instances as either bot:Zone or bot:Element, BEO specializes bot:Element to more types of building elements such as beo:Column.

The converter is an online tool where the user can upload an IFC file, and an RDF file is automatically generated. Officially, KGG only supports the IFC4 schema, whereas the construction industry often uses earlier IFC schema versions. In practice, we observed that it also supports IFC4_ADD2 models. No 3D geometric data are included in the final RDF.

Table 1 summarizes some important features of these converter tools, namely the considered version (the last one available at the moment of submitting the paper), programming language, supported Application Programming Interfaces (API), supported Operating Systems (OS), interfaces (e.g. Command Line Interface or CLI, Desktop Graphic User Interface or Desktop GUI, and Web), origin (original institutions of the developers), license, and price.

Table 2 shows some semantic information about each converter tool, namely supported reference ontology schema, supported input IFC schema, and supported output RDF serializations. One can see that there is no standard ontology that is widely supported by all tools. In particular, the outputs of IFCtoLBD and KGG do not use ifcOWL.

The four converter tools were publicly available and could be used freely at the time of doing our experiments. They can all be used from Java applications, which is convenient, for example, to combine them with the OWL API or with semantic reasoners with a Java interface (such as HermiT, Pellet, or TrOWL). However, there are notable differences in the supported versions of the IFC and ontology schemes.

5. Methodology

This section describes our methodology, designed to answer our research questions. It starts by describing the datasets and the hardware and software configuration used in the experiments. Then, it describes how the different research questions have been addressed.

Essentially, we follow a *mixed-methods approach*, which considers both quantitative and qualitative data [45]. For example, our measurable variables include quantitative data such as the running time of the experiments and qualitative data such as our perception about the quality of

[28] BOT V0.3.2 has 7 classes, 14 object properties, and 1 data property.
[29] https://www.student.dtu.dk/~mhoras/presentations/defense.html#/24 (examples).
[30] Such links are called predicates by the authors of IFCtoLBD.
[31] https://github.com/jyrkioraskari/IFCtoLBD/releases/tag/2.43.5, last version updated.
[32] Before version V2.40.1, geometric data was always omitted.
[33] https://github.com/kyriakos-katsigarakis/openmetrics/tree/master/openmetrics-kgg
[34] https://kgg.openmetrics.eu (No longer available, "Error 502 Bad Gateway").
[35] https://brickschema.org

Table 2
Ontology schema, IFC schema, and RDF serializations supported by converter tools. ifcOWL-Standard is the OWL 2 RL version of ifcOWL, and ifcOWL-Extended is the name of the OWL 2 DL version used by IFC2LD.

Converter tool	Ontology schema	IFC schema	RDF serialization
IFC2LD V1.5	ifcOWL-Standard, ifcOWL-Extended	IFC2X3_TC1, IFC4, IFC4_ADD1	TURTLE, TURTLE.GZ, NTRIPLES NQUADS, RDFXML, JSONLD
IFCtoRDF (CLI V0.4 and Desktop V2.8.1)	ifcOWL	IFC2X3_TC1, IFC4, IFC4_ADD1, IFC4_ADD2,	TURTLE
IFCtoLBD V2.43.5	BOT, PRODUCT, PROPS, ifcOWL, OMG, OPM	IFC4_ADD2_TC1, IFC4X1, IFC4X3_RC1	
KGG V0.2.0	BRICK, BEO, BOT, PROPS	IFC4	

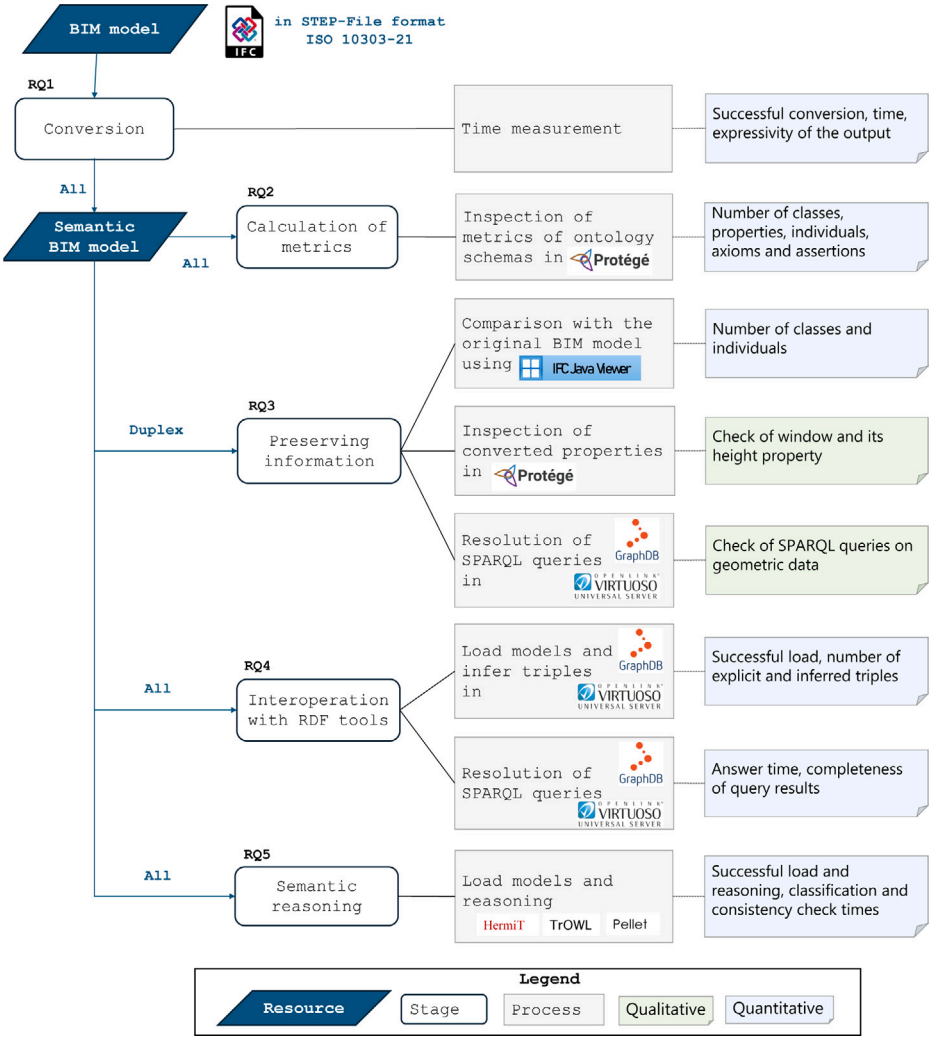


Fig. 3. Summary of our methodology.

the translations. The setup of our experiments are described in detail for the sake of reproducibility, and some steps are repeated several times for the sake of variability.

Fig. 3 summarizes our methodology. It shows the procedures and processes that we designed to obtain some qualitative and quantitative measures whose analysis can help to measure computation times for common tasks, evaluate knowledge retention, examine reasoning capabilities, and, ultimately, to answer our research questions.

Datasets. Our evaluation was performed over four datasets (see details in Table 3) that were obtained from three different BIM models represented as IFC files. Fig. 4 shows the 3D models visualized on Solibri.

Duplex. It is a publicly available BIM model that represents a duplex apartment.[36]

[36] https://github.com/MadsHolten/BOT-Duplex-house (in *"Model files"* folder).



a) b) c)

Fig. 4. BIM models: (a) Duplex, (b) Schependomlaan, and (c) 7-dwelling building.

Table 3
BIM models used in our test.

BIM model	Size (MB)	IFC schema
Duplex	2.7	IFC2X3
Schependomlaan	40	
7-dwelling building	11.2	
	10	IFC4_ADD2

Schependomlaan. It is a public BIM dataset.[37] This project was developed and built by Hendriks Bouw en Ontwikkeling and comprises 10 apartments located in Nijmegen (The Netherlands). The dataset contains a design model in IFC including planning, suppliers, point clouds, schedules, and construction log files.

7-dwelling building. It is a BIM model owned by the EiPM company.[38] It represents a real building located in the Barcelona area. The model has 4 building storeys and 7 dwellings: 2 of them on the first floor and 5 on the second one. This dataset has previously been used for compliance checking of habitability regulation [15]. There are two versions of this dataset, using IFC2X3 and IFC4_ADD2 schemes. Both of them were used in the experiments.

IFC2LD, IFCtoRDF, and IFCtoLBD were evaluated using all these datasets. However, KGG was only evaluated with the 7-dwelling building, because it is the only dataset in the IFC schema version (IFC4_ADD2) supported by the tool.

Hardware, software versions and configuration. We will first report the version and the configuration of each converter tool that was used in the experiments. The Base URI was http://sid.cps.unizar.es/converters.owl/. In IFC2LD V1.5, three options for the layers were tried: *Simple*, *Standard*, and *Extended*. In IFCtoRDF V2.8.1, the option to create GUID[39] URIs were enabled. Finally, in IFCtoLBD Desktop V2.43.5 several cases were tried: the three complexity levels of PROPS ($L1$, $L2$, and $L3$) were tried, both with and without "Blank nodes", and enabling both "Generate geometry" and "Performance optimization".

Now, we enumerate the other software tools used in our experiments. The popular ontology editor Protégé V5.5.0 was used to visualize the OWL 2 ontologies [46].[40] To manage ontologies, OWL API V3.4.10 was used from a Java program that loads an OWL 2 ontology and calls a semantic reasoner to solve two reasoning tasks: classification and consistency. For ontology reasoning, three classical reasoners, namely HermiT V1.3.8, TrOWL V3.4, and Pellet V2.3.3, were considered, as all of them fully support OWL 2 DL ontologies. To solve SPARQL queries and for RDF graph visualization, the free versions of Virtuoso V8.3 and GraphDB V10.0 were used (Apache Jena Fuseki V3.14.0 was also used in some tests).

Finally, regarding the environment used in the experiments, the evaluation was performed on a laptop Intel Core i7-1165G7 1.69 GHz, 16 GB RAM (8 GB were allocated for the Java Virtual Machine). The operating system was Windows 10 pro 64-bit and an Internet connection was needed to evaluate KGG.

Conversion time and robustness. To answer RQ1, our experiment comprises the conversion of the datasets using the 4 converter tools and different configurations, which results in 18 possible cases. In each case, the output files, some possible errors in the conversion, and the translation times were computed. Experiments were repeated three times to make the evaluation more robust and the average time was computed.

IFCtoLBD V2.43.5 was tested with and without blank nodes in all its configurations, but the same conversion times and number of triples were obtained using both configurations (indeed, no blank nodes were found in the output). Therefore, in the following, the text will be restricted to the case without blank nodes.

Output metrics. To answer RQ2, several ontology metrics about the converted BIM models were analyzed, as well as some metrics of the ontology schemes needed for the different converter tools. In particular, the experiment consists of loading each converted BIM model into the Protégé ontology editor and retrieving some ontology metrics, namely the total number of Classes, Object Properties, Data Properties, Individuals, Axioms, Class Assertions, Object Property Assertions, Data Property Assertions, and expressivity (Description Logic language). Before doing that, the output files were edited to ensure that they input their respective ontology schemes.

[37] https://github.com/openBIMstandards/DataSetSchependomlaan
[38] http://www.eipm.es
[39] A GUID (Globally Unique IDentifier) is a universally unique identifier so that elements in different projects cannot have the same ID.
[40] http://protege.stanford.edu

```
#6921=IFCWINDOW('1hOSvn6df7F8_7GcBWlRLx',#33,'M_Fixed:750mm x 2200mm:750mm x 2200mm:146885'
      ,$,'750mm x 2200mm',#6920,#6915,'146885',2.199999999999996,0.7499999999999996);
...
#6940=IFCPROPERTYSINGLEVALUE('Height',$,IFCLENGTHMEASURE(2.2),$);
...
#6993=IFCPROPERTYSET('3wXESTX8r65A35RpVNAwPp',#33,'PSet_Revit_Type_Dimensions',$,
      (#6940,#6942,#6943,#6944));
...
#38580=IFCRELDEFINESBYPROPERTIES('002GW$n7bBQR51XrlqHj8z',#33,$,$,
      (#6921,#7025,#22396,#22448),#6993);
```

Fig. 5. Extract of Duplex IFC file.

Preserving information. The next step is to compare the result of the converter tools with the original IFC models, to answer research question RQ3. To do so, three different tasks were identified: comparing the preservation of classes and individuals, comparing the preservation of properties, and comparing the preservation of geometric information.

The number of classes and individuals in the original IFC model can be partially measured using some BIM software, and then they can be compared with the RDF-converted files. In particular, IFC Java viewer 4.5.0[41] was used to estimate the number of classes by counting the different number of values of ObjectType in the model. Then, both IFC Java viewer and Solibri Anywhere 9.12.7[42] were used to measure the number of individuals.

Unfortunately, to the best of our knowledge, there is no automatic way to retrieve the number of properties and axioms. Computing those values manually is not a feasible option, as there are thousands of them. Instead, the experiment involves the visualization of the converted files (in .ttl format) using the Protégé ontology editor and the study of how different converter tools manage properties. In order to be more illustrative, a single case will be considered: how to represent a window individual (with GUID *"1hOSvn6df7F8_7GcBWlRLx"*) and its height attribute, from the Duplex BIM model. The Duplex dataset is considered because all converter tools, with the exception of KGG, supported it. Fig. 5 shows an extract of the Duplex IFC model that defines an instance with ID=#6921 of the IfcWindow class. The entity with ID=#6940 has a property (Height) and a set of properties (ID=#6993) which is linked to the instance with ID=#6921 in the IfcRelDefinesByProperties (ID=#38580).

The geometric (3D) data and properties of the building elements are necessary for construction applications. While the BIM model offers a complete geometrical representation, this is different in the converted RDF graphs or reference ontologies. Similarly, as with properties, our experiment involves the visualization of the converted files (in .ttl format) with Protégé. As an example, a SPARQL query will be considered: retrieving from the Duplex BIM model the individuals of IfcWindow class, browsing from objectPlacement to coordinates properties to get the numerical values of the Cartesian coordinates.

Interoperation with RDF tools. To answer RQ4, how to reason using graph models must be addressed. Firstly, the numbers of explicit and inferred RDF triples were analyzed, to evaluate the completeness. The experiment consists of uploading BIM models converted by each converter tool (.ttl files[43]) to two graph databases, GraphDB and Virtuoso. Only successfully completed conversions are considered. Next, the number of RDF triples in every file was queried and the following values were computed: the number of explicit triples (# Exp.), the number of inferred triples (# Inf.), and the sum of them, i.e., the total number of triples (Total). To obtain the inferred number of triples in Virtuoso, "Reasoning" mode was enabled[44]; GraphDB has the inference mode enabled by default.

Then, the answering time of SPARQL queries was evaluated. For a set of SPARQL queries defined by our research team (detailed in Appendix C) to retrieve window individuals and some of their attributes, the answering time was computed using Virtuoso and GraphDB. Experiments were repeated three times and the average values were computed. For the Duplex, Schependomlaan, and 7-dwelling (IFC2X3) datasets, the height and width of the windows were retrieved using IFC2LD, IFCtoRDF, and IFCtoLBD. For the 7-dwelling building (IFC4_ADD2), the area was retrieved using IFCtoLBD and KGG.[45]

Semantic reasoning. Finally, to answer RQ5, an experiment was designed to explore whether the converters are appropriate for semantic reasoners to retrieve implicit knowledge. Each of the outputs of the converter tools was tested with three semantic reasoners: HermiT, TrOWL, and Pellet. The times (in seconds) to load and classify the ontology and to check the consistency were computed. Note that many semantic queries (i.e., retrieving the subclasses or individuals of a class) are solved immediately once the ontology is classified [32]. The maximum possible memory and a timeout of two hours to solve those tasks were used. Experiments were repeated three times and average times were computed.

6. Results

This section reports the result of our experiments. Firstly, Section 6.1 describes our experiments about the conversion time (RQ1). Secondly, Section 6.2 discusses different ontology metrics of the converted files (RQ2). Then, Section 6.3 compares the converted and the original models (RQ3). Next, Section 6.4 measures the number of explicit and implicit RDF triples and evaluates the querying time using SPARQL engines (RQ4). Finally, Section 6.5 reports our experiments regarding the reasoning time when using semantic reasoners (RQ5).

6.1. Conversion time and robustness

Table 4 includes errors found during the conversions of the files. For the successful cases, Table 5 shows the conversion time (in seconds) of every BIM model and converter tool.

It is worth to note that IFC2LD failed in all our tests to adapt the ifcOWL ontology to the IFC EXPRESS schema of the input file, so we had to manually submit it.

[41] http://www.apstex.com

[42] https://www.solibri.com

[43] A file with the shortest conversion time was uploaded.

[44] https://docs.openlinksw.com/virtuoso/rdfsparqlruleenableinfr/, using the option define input:inference "http://www.w3.org/2002/07/owl#".

[45] Recall that KGG does not represent height and width.

Table 4
Errors found during the translation from IFC to RDF.

Converter	Dataset	Configuration	Message
IFC2LD	All	"Simple" layer case using both uppercase and lowercase letters	*"Unexpected error: Cannot find complex processor by name 'Simple'"*
IFC2LD	Schependomlaan	"Standard" and "Extended"	*"Java heap space error"*
IFC2LD	7-dwelling building (IFC2X3)	"Extended" layer	*"Assertion error: convertLiteralToNode"*
IFC2LD	7-dwelling building (IFC4_ADD2)	"Standard" and "Extended"	*"Entity type not found: 'IFCCARTESIANPOINTLIST2D' "*
IFCtoRDF	7-dwelling building (IFC4_ADD2)	It saved an empty file	None
KGG	All except 7-dwelling (IFC4_ADD2)	All	*"Unsupported IFC schema"*

Table 5
Conversion time (s) of every BIM model and converter tool version.

#	Converter tool	Duplex	Schependomlaan	7-dwelling (IFC2X3)	7-dwelling (IFC4_ADD2)
1	IFC2LD V1.5 (Simple)	Error	Error	Error	Error
2	IFC2LD V1.5 (Standard)	**4.48**	Error	14.07	Error
3	IFC2LD V1.5 (Extended)	4.62	Error	Error	Error
4	IFCtoRDF V0.4	6.30	**23.27**	**11.68**	Error
5	IFCtoRDF V2.8.1	9.83	62.85	26.61	Error
6	IFCtoLBD V2.43.5 (BOT)	49.33	103.18	76.79	134.94
7	IFCtoLBD V2.43.5 (BOT+PRODUCT)	50.55	121.58	80.00	124.72
8	IFCtoLBD V2.43.5 (BOT+PROPS L1)	50.19	124.59	84.13	137.88
9	IFCtoLBD V2.43.5 (BOT+PROPS L2)	53.96	145.74	79.07	131.51
10	IFCtoLBD V2.43.5 (BOT+PROPS L3)	52.83	171.97	86.61	144.12
11	IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	70.59	139.99	86.79	131.54
12	IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	55.10	146.96	85.10	141.06
13	IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	53.75	160.14	89.37	136.00
14	KGG V0.2.0	Error	Error	Error	**48.63**

Table 6
Ontology schema metrics.

Ontology	#C	#OP	#DP	#I	#A	URI prefix
ifcOWL-Standard	1,085	1,059	568	1,915	7,642	http://drumbeat.cs.hut.fi/owl/IFC2X3_Standard#
ifcOWL-Extended	1,137	1,060	568	1,015	9,730	http://drumbeat.cs.hut.fi/owl/IFC2X3_Extended#
ifcOWL	1,093	1,422	5	1,018	17,782	https://standards.buildingsmart.org/IFC/DEV/IFC2x3/TC1/OWL#
	1,315	1,589	0	1,159	20,640	https://standards.buildingsmart.org/IFC/DEV/IFC4/ADD2/OWL#
BOT	10	16	1	5	897	https://w3id.org/bot#
BEO	186	0	1	1	1,516	https://pi.pauwel.be/voc/buildingelement#
MEP	484	0	1	1	3,527	https://pi.pauwel.be/voc/distributionelement#
PROPS	2	3,977	0	0	19,881	http://lbd.arch.rwth-aachen.de/props#
OMG	8	17	2	0	130	https://w3id.org/omg#
OPM	17	8	4	1	142	https://w3id.org/opm#
BRICK	837	24	0	1,252	9,665	https://brickschema.org/schema/1.1/Brick#

6.2. Output metrics

Table 6 shows, for each ontology schema (ifcOWL, BOT, BEO, MEP, PROPS, and BRICK), the total number of Classes (C), Object Properties (OP), Data Properties (DP), Individuals (I), Axioms (A), and the URI prefix. Prefixes shown in the table are those used by the converted tools.

Note that IFC2LD and IFCtoRDF use different versions of the ifcOWL ontology, as IFC2LD defines subontologies with different expressivities (Standard or Extended modes.[46]) Furthermore, in the case of IFCtoRDF, there are two different prefixes, depending on the IFC Express Schema of the input ontology. In IFC2LD, this is not the case, because it only supported our IFC2X3 datasets. Furthermore, the PRODUCT ontology is replaced by its two modular subontologies BEO and MEP.

Table 7 shows the metrics for Duplex, Table 8 for Schependomlaan, Table 9 for 7-dwelling building (IFC2x3), and Table 10 for 7-dwelling building (IFC4_ADD2). In particular, the tables show the total number of Classes (C), Object Properties (OP), Data Properties (DP), Individuals (I), Axioms (A), Class Assertions (CA), Object Property Assertions (OPA), and Data Property Assertions (DPA). Given that every converter tool generated output files with a different number of triples, it is evident that the ontology metrics will also be different.

During the importation of the schemes, we found that IFCtoLBD V0.1 and KGG V0.2.0 use the URI ⟨https://w3id.org/props#⟩ for PROPS, which is not a valid URL.[47] Instead, to use the PROPS schema, IFCTOLBD V2.43.5 outputs import a different URI ⟨http://lbd.arch.rwth-aachen.de/props#⟩.

Furthermore, Table 11 shows the expressivity of the outputs of the different converter tools, for each different configuration. The expressivity was always the same for all the tested datasets.

[46] Recall that there is a Simple mode, but it did not work, as discussed in Section 5.
[47] *"HTTP Error 443: The requested resource is not found"*.

Table 7
Duplex data results of every converter tool.

Converter tool	Duplex							
	#C	#OP	#DP	#I	#A	#CA	#OPA	#DPA
IFC2LD V1.5 (Standard)	1,085	1,059	**568**	4,908	140,829	44,401	63,233	**15,503**
IFC2LD V1.5 (Extended)	1,137	1,060	**568**	4,908	**304,456**	58,887	**128,367**	**15,503**
IFCtoRDF V0.4	**1,222**	1,422	5	**81,901**	244,925	**82,196**	64,241	11,450
IFCtoRDF V2.8.1	**1,222**	1,422	5	**81,901**	244,925	**82,196**	64,241	11,450
IFCtoLBD V2.43.5 (BOT)	10	17	7	722	5,880	254	720	1,416
IFCtoLBD V2.43.5 (BOT+PRODUCT)	677	17	8	724	11,186	525	720	1,422
IFCtoLBD V2.43.5 (BOT+PROPS L1)	11	3,991	254	722	51,252	254	720	7,292
IFCtoLBD V2.43.5 (BOT+PROPS L2)	11	4,209	7	6,842	68,636	6,374	6,840	1,416
IFCtoLBD V2.43.5 (BOT+PROPS L3)	12	**4,210**	7	12,969	87,024	12,501	12,967	1,416
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	678	3,994	255	724	36,677	525	720	7,298
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	678	4,209	8	6,844	54,094	6,645	6,840	1,422
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	679	4,210	8	12,971	72,482	12,772	12,967	1,422

Table 8
Schependomlaan data results of every converter tool.

Converter tool	Schependomlaan							
	#C	#OP	#DP	#I	#A	#CA	#OPA	#DPA
IFCtoRDF V0.4	**1,239**	1,422	5	**922,314**	**2,903,446**	**922,609**	**801,735**	98,384
IFCtoRDF V2.8.1	**1,239**	1,422	5	922,313	2,903,441	**922,609**	**801,735**	98,384
IFCtoLBD V2.43.5 (BOT)	10	17	7	10,628	66,345	3,624	7,178	21,024
IFCtoLBD V2.43.5 (BOT+PRODUCT)	691	16	8	57,983	216,995	57,829	57,325	4,164
IFCtoLBD V2.43.5 (BOT+PROPS L1)	11	3,994	840	10,628	249,759	3,624	7,178	167,697
IFCtoLBD V2.43.5 (BOT+PROPS L2)	11	4,811	7	160,916	693,140	153,912	157,466	21,024
IFCtoLBD V2.43.5 (BOT+PROPS L3)	12	4,812	7	314,771	1,158,272	307,767	311,321	21,024
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	678	3,994	**841**	10,630	240,159	8,870	7,178	**167,703**
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	678	4,811	8	160,918	683,557	159,158	157,466	21,030
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	692	**4,819**	11	314,774	1,148,831	313,018	311,321	21,034

Table 9
7-dwelling building (IFC2X3) data results of every converter tool.

Converter tool	7-dwelling building (IFC2X3)							
	#C	#OP	#DP	#I	#A	#CA	#OPA	#DPA
IFC2LD V1.5 (Standard)	1,085	1,059	**568**	25,346	641,221	208,226	303,319	**73,727**
IFCtoRDF V0.4	**1,239**	1,422	5	**397,117**	**1,138,166**	397,412	311,196	47,128
IFCtoRDF V2.8.1	**1,239**	1,422	5	**397,117**	**1,138,166**	397,412	311,196	47,128
IFCtoLBD V2.43.5 (BOT)	10	17	7	2,113	17,196	717	1,402	4,200
IFCtoLBD V2.43.5 (BOT+PRODUCT)	677	17	8	2,115	22,982	1,468	1,592	4,206
IFCtoLBD V2.43.5 (BOT+PROPS L1)	19	3,994	237	2,143	82,618	747	1,492	30,507
IFCtoLBD V2.43.5 (BOT+PROPS L2)	19	4,192	7	28,923	162,556	27,527	28,272	4,200
IFCtoLBD V2.43.5 (BOT+PROPS L3)	20	**4,193**	7	59,120	256,311	**57,724**	**58,429**	4,200
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	686	3,994	238	2,145	67,523	1,498	1,492	30,513
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	686	4,192	8	28,925	148,494	28,278	28,272	4,206
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	687	**4,193**	8	59,122	242,249	58,475	58,469	4,206

Table 10
7-dwelling building (IFC4_ADD2) data results of every converter tool.

Converter tool	7-dwelling building (IFC4_ADD2)							
	#C	#OP	#DP	#I	#A	#CA	#OPA	#DPA
IFCtoLBD V2.43.5 (BOT)	10	17	7	2,092	16,155	710	1,466	4,158
IFCtoLBD V2.43.5 (BOT+PRODUCT)	682	17	8	2,094	22,420	1,940	1,466	4,164
IFCtoLBD V2.43.5 (BOT+PROPS L1)	18	3,994	235	2,122	80,308	740	1,466	30,100
IFCtoLBD V2.43.5 (BOT+PROPS L2)	18	4,187	7	28,691	159,963	27,309	28,035	4,158
IFCtoLBD V2.43.5 (BOT+PROPS L3)	19	4,188	7	57,981	**250,456**	56,599	**57,325**	4,158
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	690	3,994	**236**	2,124	66,692	1,970	1,466	**30,106**
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	690	4,187	8	28,693	146,383	28,539	28,035	4,164
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	704	**4,196**	11	**57,984**	237,018	**57,834**	57,325	4,168
KGG V0.2.0	**1,032**	40	2	1,733	14,881	1,476	4,539	4

6.3. Preserving information

6.3.1. Preserving classes and individuals

Table 12 shows the number of classes identified using IFC Java viewer, and the lower bound of the number of classes according to the converter tools. Unfortunately, the BIM software only identifies a small number of classes, so this does not give significant results. While IFC Java viewer

Table 11
DL expressivity of the output ontology for every converter.

#	Converter tool	DL expressivity
2	IFC2LD V1.5 (Standard)	$\mathcal{ALCOIF}$(**D**)
3	IFC2LD V1.5 (Extended)	$\mathcal{ALCOIN}$(**D**)
4	IFCtoRDF V0.4	$\mathcal{AL}$(**D**)
5	IFCtoRDF V2.8.1	$\mathcal{AL}$(**D**)
6	IFCtoLBD V2.43.5 (BOT)	$\mathcal{SRI}$(**D**)
7	IFCtoLBD V2.43.5 (BOT+PRODUCT)	$\mathcal{SRI}$(**D**)
8	IFCtoLBD V2.43.5 (BOT+PROPS L1)	$\mathcal{SRIF}$(**D**)
9	IFCtoLBD V2.43.5 (BOT+PROPS L2)	$\mathcal{SRIF}$(**D**)
10	IFCtoLBD V2.43.5 (BOT+PROPS L3)	$\mathcal{SRIF}$(**D**)
11	IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	$\mathcal{SRIF}$(**D**)
12	IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	$\mathcal{SRIF}$(**D**)
13	IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	$\mathcal{SRIF}$(**D**)
14	KGG V0.2.0	$\mathcal{SROI}$(**D**)

Table 12
Number of classes according to the BIM software and to the converter tools.

Dataset	BIM software	Converter tools
Duplex	17	1,222
Schependomlaan	20	1,239
7-dwelling building (IFC2X3)	19	1,239
7-dwelling building (IFC4_ADD2)	20	1,032

Table 13
Number of individuals according to the BIM software and to the converter tools.

Dataset	BIM software	IFC2LD	IFCtoRDF	IFCtoLBD	KGG
Duplex	287	287	287	295	
Schependomlaan	3,803		3,808	7,296	
7-dwelling building (IFC2X3)	816	844	843	1,240	
7-dwelling building (IFC4_ADD2)	779			1,199	882

```
#258648= IFCLOCALPLACEMENT(#155,#258647);
...
#258688= IFCPRODUCTDEFINITIONSHAPE($,$,(#258686,#258718));
#258691= IFCSLAB('2pXMcyfWv9igGhNWUPBtLs',#42,'Floor:Paviment Morter-Acabat Interior-5cm:1010345',
    $,'Floor:Paviment Morter-Acabat Interior-5cm',#258648,#258688,'1010345',.FLOOR.);
```

Fig. 6. Extract of 7-dwelling building (IFC4_ADD2) IFC file.

makes it possible to retrieve the number of individuals in a class, this strategy is also not reliable because most of the classes are missing (so their number of individuals cannot be known). Solibri Anywhere did not identify more classes or individuals.

Table 13 shows the number of individuals for each dataset both using IFC Java viewer and the converter tools. In IFCtoLBD, we checked that the number of individuals is the same for levels $L1$, $L2$, and $L3$.

We noticed an error in KGG when converting the 7-dwelling (IFC4_ADD2) dataset: while in the original model IfcWall has 177 individuals, in KGG the equivalent class beo:Wall has 238 individuals. The reason is that some instances of IfcSlab are wrongly considered as walls, as shown in Example 1.

Example 1. While Fig. 6 shows a building element with ID=#258691 which is an instance of IfcSlab (highlighted in orange), it is translated by KGG as an instance of beo:Wall.

6.3.2. Preserving property types

This section will use snapshots from Protégé to illustrate the different converter outputs. Appendix B shows examples of RDF triples.

Fig. 7 corresponds to the IFC2LD V1.5 converter and shows an instance of the IfcWindow class, with some object properties and data properties. overallHeight, representing property Height in the IFC model, is correctly represented as a data property.[48]

Note that the object properties are related to anonymous individuals (e.g., objectPlacement is related to _:genid2147502103) and it is not possible to access their values, i.e., one cannot click on the value and see their properties and values directly.

Fig. 8 shows the IFCtoRDF V2.8.1 converter output, although both versions of this converter offer a similar result (from a visualization point of view). Now, Height is represented using an object property overallHeight_IfcWindow which links a window and an individual (inst:IfcPositiveLengthMeasure_46591) representing the measure, and this individual is then linked to the real height value. However, such properties are interpreted as annotations, making it impossible to access the real height value (‘‘2.199999999999996’’^^xsd:double).

Fig. 9 shows the IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS Level $L1$) converter. Interestingly, in IFCtoLBD V2.43.5, there are some examples of two different properties representing the same thing. For example, a link between an individual and its height is represented using both

[48] In Protégé, object datatype, data properties, and annotation properties are represented using blue, green, and orange rectangles, respectively.

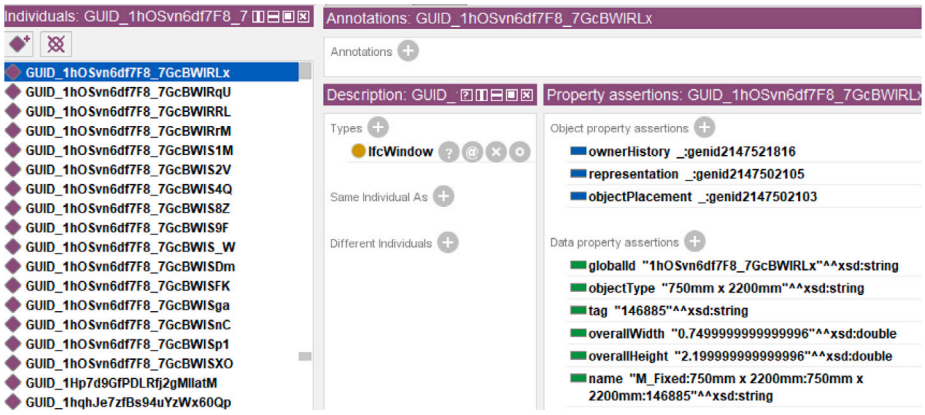


Fig. 7. Screenshot of IFC2LD V1.5.

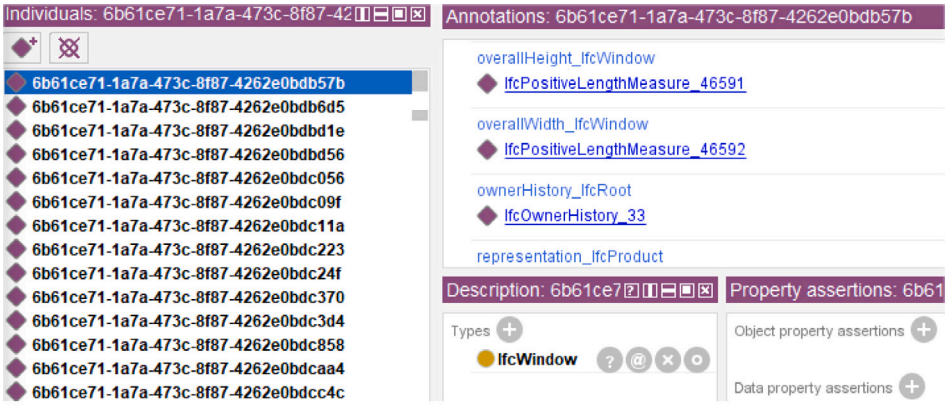


Fig. 8. Screenshot of IFCtoRDF V2.8.1.

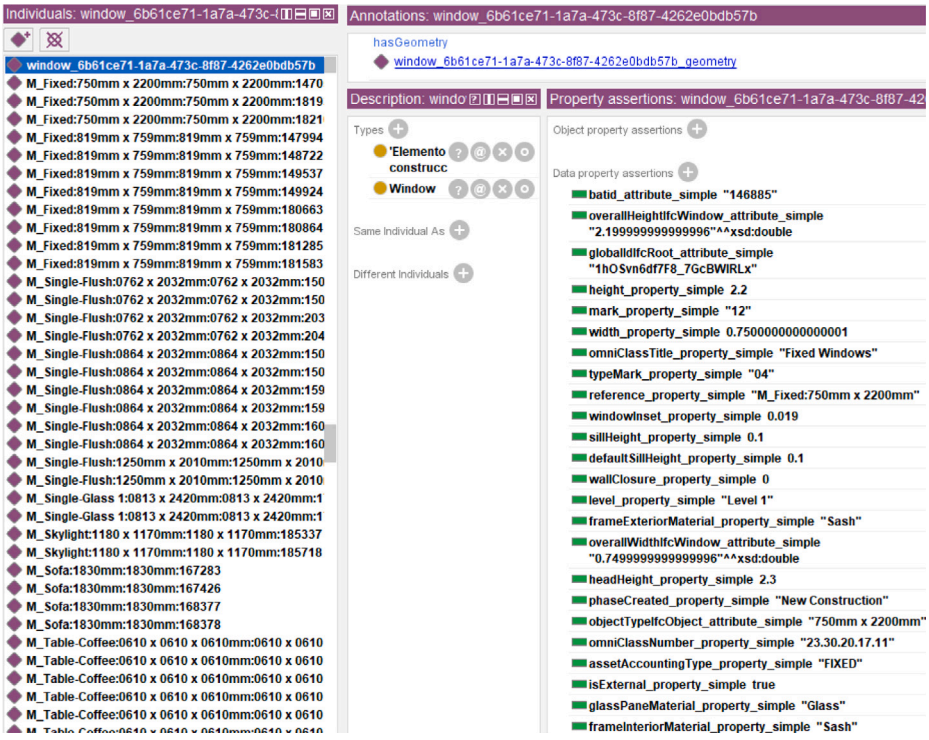


Fig. 9. Screenshot of IFCtoLBD V2.43.5 result (Level *L*1).

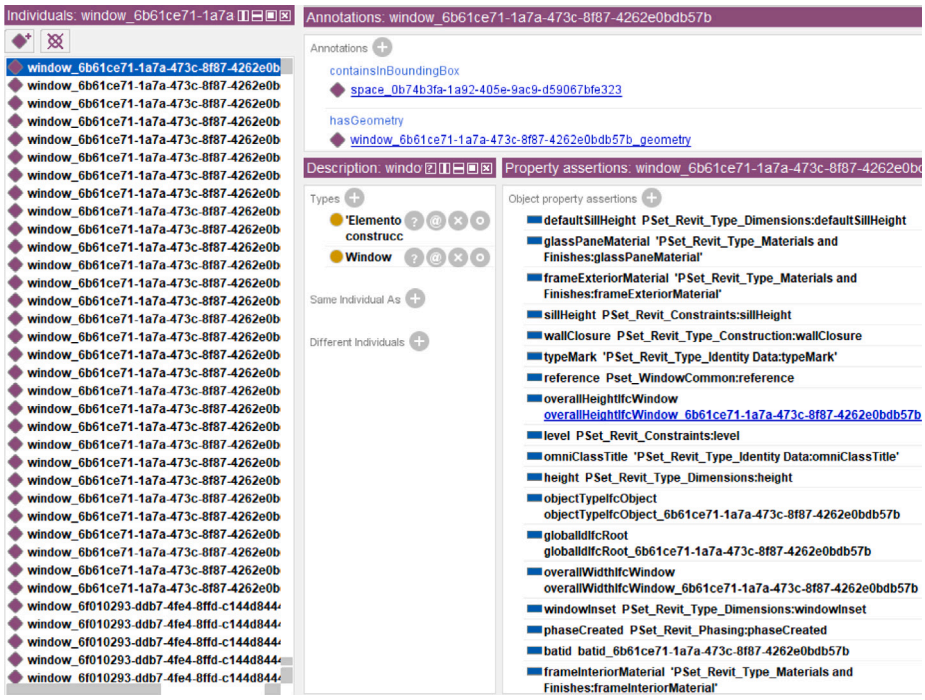


Fig. 10. Screenshot of IFCtoLBD V2.43.5 result (Level *L*2).

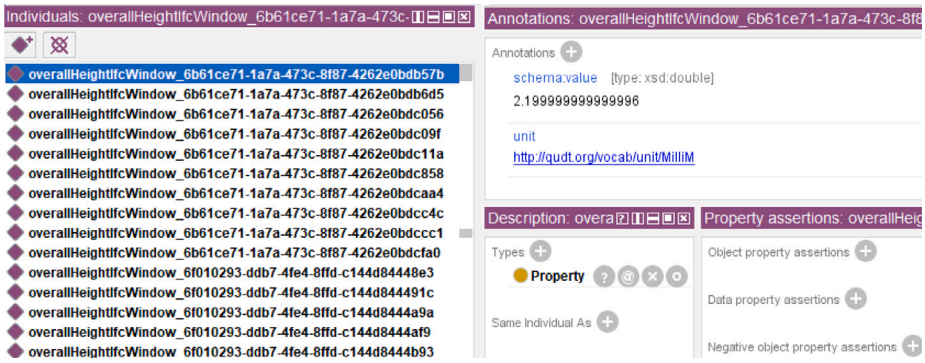


Fig. 11. Screenshot of IFCtoLBD V2.43.5 result (Level *L*2) for overallHeightIfcWindow property.

overallHeightIfcWindow_attribute_simple and height_property_simple. Now, properties with a numerical range are represented using data properties. For instance, although in the PROPS ontology schema, height and height_simple are defined as object properties, IFCtoLBD uses instead the data properties overallHeightIfcWindow_attribute_simple and height_property_simple, not defined in PROPS. Instead, the annotation property hasGeometry is used to link geometric data.

Fig. 10 shows the result obtained by using IFCtoLBD V2.43.5 with the configuration BOT+PRODUCT+PROPS Level *L*2. Now, there are some object properties and some annotation properties. For example, height and overallHeightIfcWindow are object properties (it is worth to recall that height is defined at PROPS schema and that overallHeightIfcWindow was a data property in level *L*1). Both properties have the same purpose: associating a window with an individual (in the example, overallHeightIfcWindow_ 6b61ce71-1a7a-473c-8f87-4262e0bdb57b) which is then linked with a numerical value via schema:value, i.e., there is a path of length two: from the window to the intermediate individual using an object property, and from the intermediate individual to the numerical value using a data property. Now, Protégé interprets the numerical value as an annotation (see Fig. 11).

However, we noticed that Protégé does not display properly the numerical values of the height property. For Example 7, the value should be 2.2 but Protégé displays 0.61, as shown in Fig. 12. Rather than linking directly the intermediate individual with an annotation value (as in Fig. 11), Protégé adds an intermediate step, and creates an individual named PSet_Revit_Type_Dimensions:height. However, since there are many individuals with the same name, it displays the value that corresponds to another window instance.

Fig. 13 follows the example for the configuration IFCtoLBD V2.43.5 (BOT+PRO- DUCT+PROPS Level *L*3). The output is similar to that of Level *L*2. Here, the overallHeightIfcWindow object property links to an individual that has the relationship with a new individual (see Fig. 14). This new individual contains a value annotated via schema:value (see Fig. 15). Again, the values of the height property are not properly displayed.

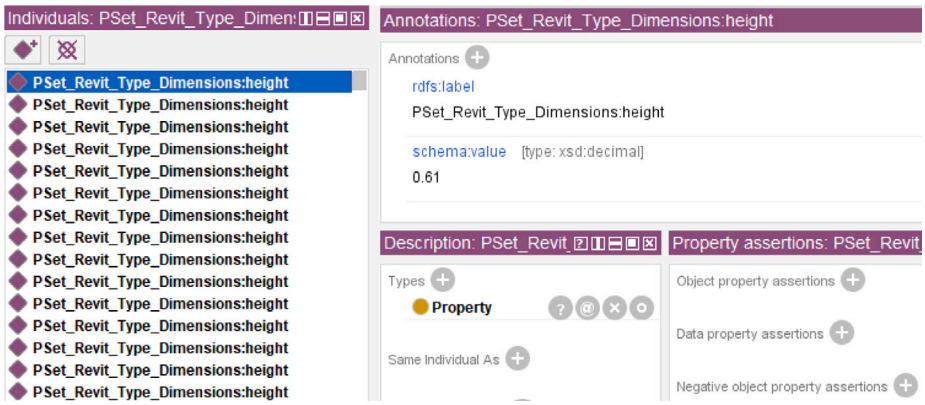


Fig. 12. Screenshot of IFCtoLBD V2.43.5 result (Level *L2*) for height property.

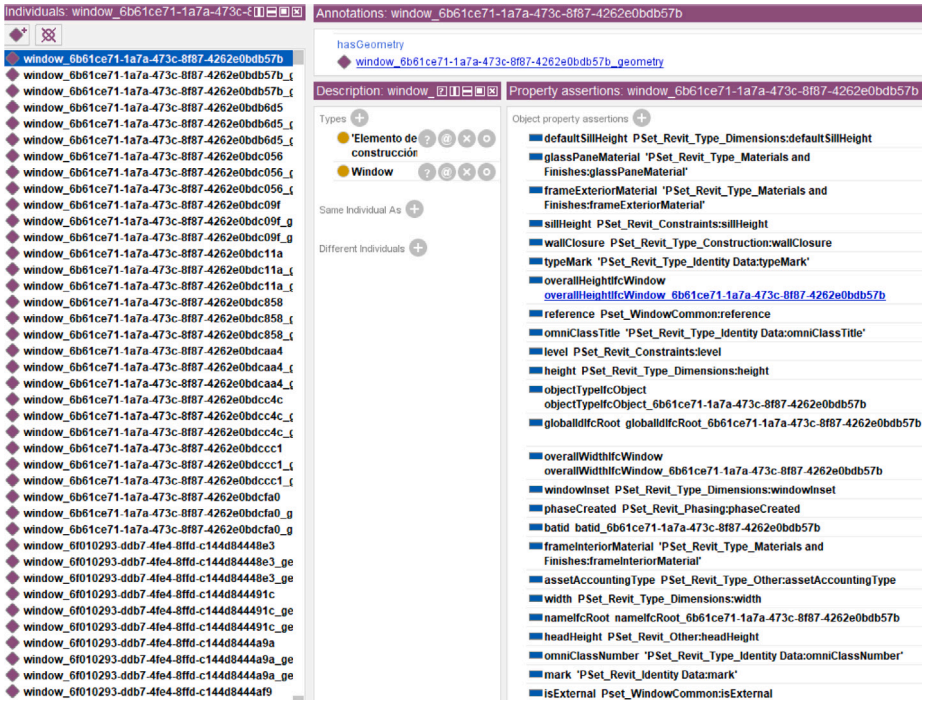


Fig. 13. Screenshot of IFCtoLBD V2.43.5 result (Level *L3*).

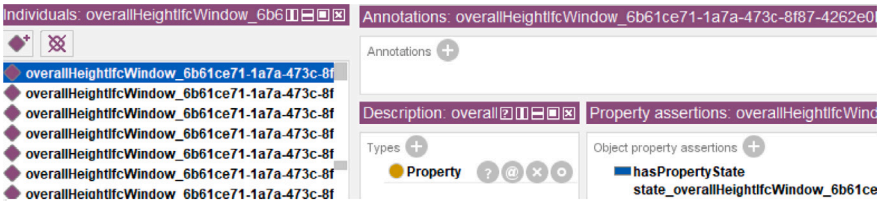


Fig. 14. Screenshot of IFCtoLBD V2.43.5 (Level *L3*): first individual for overallHeightIfcWindow property.

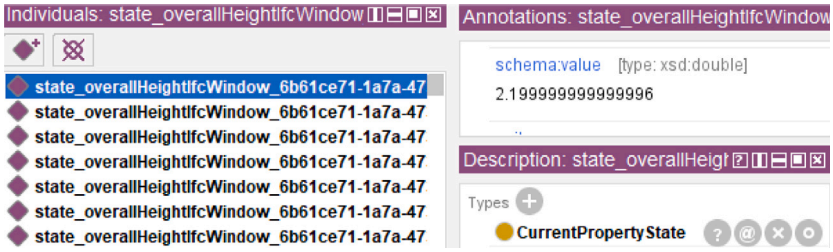


Fig. 15. Screenshot of IFCtoLBD V2.43.5 (Level *L3*): second individual for overallHeightIfcWindow property.

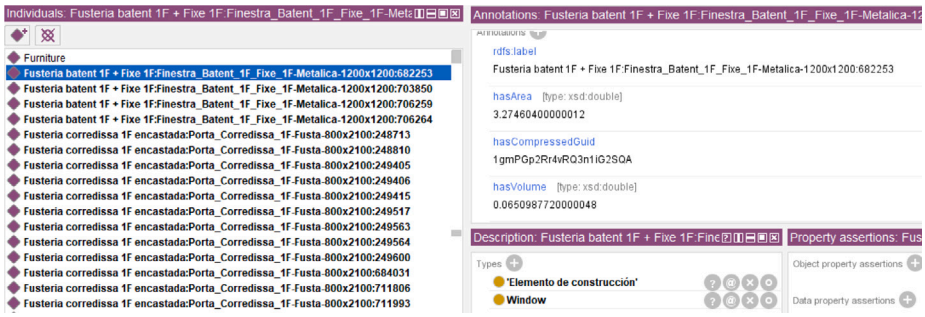


Fig. 16. Screenshot of KGG result.

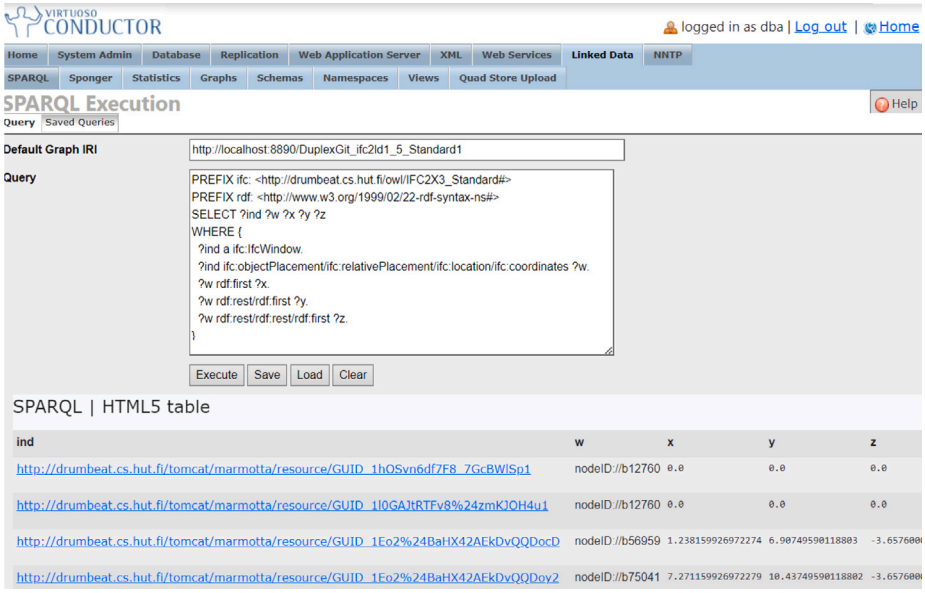


Fig. 17. IFC2LD V1.5 converter tool: geometric data of windows using Virtuoso.

Fig. 16 illustrates the KGG V0.2.0 output. The ontology editor cannot import the PROPS URIs and interprets the properties as annotations, for example, hasVolume property of a Window instance in the 7-dwelling (IFC4_ADD2) BIM model (since Duplex is not supported by KGG). It is worth mentioning that KGG computes new properties (e.g. hasArea) and omits other ones (e.g. height), although the official webpages say the opposite.[49]

As a final comment, note that all the previous tools use xsd:double to represent real numbers, although in general xsd:decimal would be preferable [47].

6.3.3. *Preserving geometric data*

IFC2LD, IFCtoRDF, and IFCtoLBD can keep the geometric data, but there are notable differences between them.

Firstly, IFC2LD V1.5 (with Standard configuration) manages the geometric data of each individual as a list (RDF collections[50]) inside of blank nodes. Now, the SPARQL query is:

```
SELECT ?ind ?w ?x ?y ?z
WHERE {
  ?ind a ifc:IfcWindow.
  ?ind ifc:objectPlacement/ifc:relativePlacement/ifc:location/ifc:coordinates ?w.
  ?w rdf:first ?x.
  ?w rdf:rest/rdf:first ?y.
  ?w rdf:rest/rdf:rest/rdf:first ?z.
}
```

The query was run on Virtuoso and GraphDB. Virtuoso and Jena returned the same results, 24, but GraphDB returned 1,060,536 results. Fig. 17 shows the query results obtained using Virtuoso: some windows, a blank node, and the x, y, and z coordinates of each window. Unfortunately, GraphDB gives a strange result, with a huge number of windows and different y coordinates, as illustrated in Fig. 18.

Recall that one cannot check the content of blank nodes through Protégé, so it is necessary to use a text editor and manually find the ifc:IfcCartesianPoint and its ifc:coordinates.

[49] https://w3c-lbd-cg.github.io/opm/#Calculations

[50] http://www.snee.com/bobdc.blog/2014/04/rdf-lists-and-sparql.html

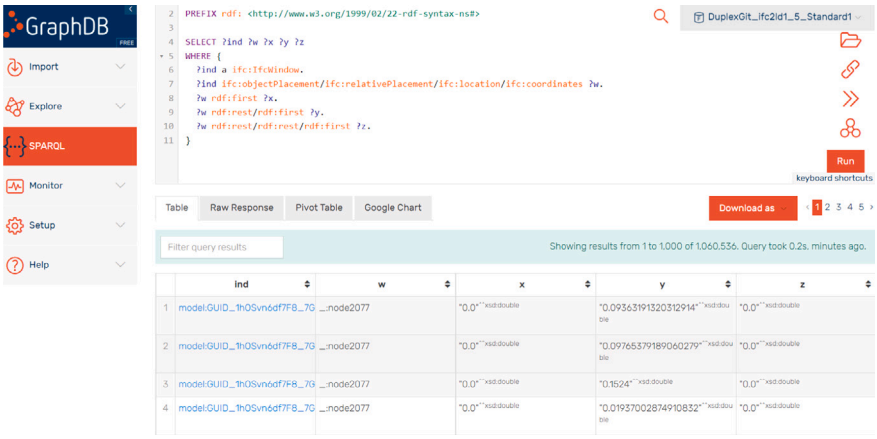


Fig. 18. IFC2LD V1.5 converter tool: geometric data of windows using GraphDB.

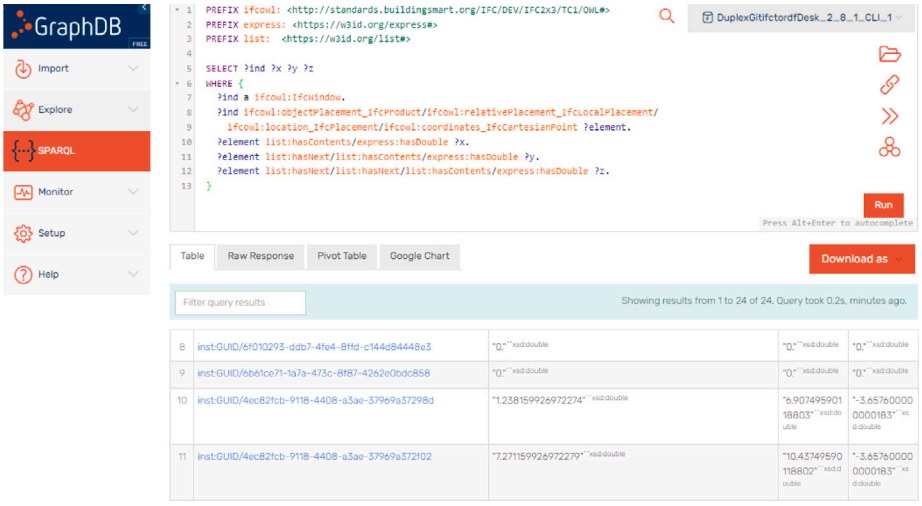


Fig. 19. IFCtoRDF converter tool: geometric data of windows using GraphDB.

Secondly, IFCtoRDF uses the general purpose `list` (defined in <https://w3id.org/list#>) to represent geometric data, as explained in [24]. The query below has a linked list (an instance serialization) to get the coordinates of ifcowl:coordinates_IfcCartesianPoint:

```
SELECT ?ind ?x ?y ?z
WHERE {
  ?ind a ifcowl:IfcWindow.
  ?ind ifcowl:objectPlacement_IfcProduct/ifcowl:relativePlacement_IfcLocalPlacement/
    ifcowl:location_IfcPlacement/ifcowl:coordinates_IfcCartesianPoint ?element.
  ?element list:hasContents/express:hasDouble ?x.
  ?element list:hasNext/list:hasContents/express:hasDouble ?y.
  ?element list:hasNext/list:hasNext/list:hasContents/express:hasDouble ?z.
}
```

In this case, both Virtuoso and GraphDB return the same number of windows (24), as expected. Fig. 19 shows the result of the previous query provided by GraphDB, with the Cartesian coordinates of some windows. IFCtoRDF V2.81 and V0.4 gave the same result. Remember that, to access geometric data from Protégé, it is necessary to browse through annotations.

Finally, IFCtoLBD V2.43.5 uses the OMG ontology to manage the geometry. In particular, the object property hasGeometry relates an object and its geometric representation, which is enclosed in a IfcBoundingBox with its local coordinate system. The SPARQL query is:

```
SELECT ?ind ?z_max ?z_min ?y_max ?y_min ?x_max  ?x_min
WHERE {
    ?ind a beo:Window.
    ?ind omg:hasGeometry/lbd:hasBoundingBox ?ind_geometry_bb.
    ?ind_geometry_bb lbd:z-max ?z_max.
    ?ind_geometry_bb lbd:z-min ?z_min.
    ?ind_geometry_bb lbd:y-max ?y_max.
    ?ind_geometry_bb lbd:y-min ?y_min.
    ?ind_geometry_bb lbd:x-max ?x_max.
    ?ind_geometry_bb lbd:x-min ?x_min.
}
```

As in the previous case, both Virtuoso and GraphDB return the same number of windows (24), as shown in Fig. 20.

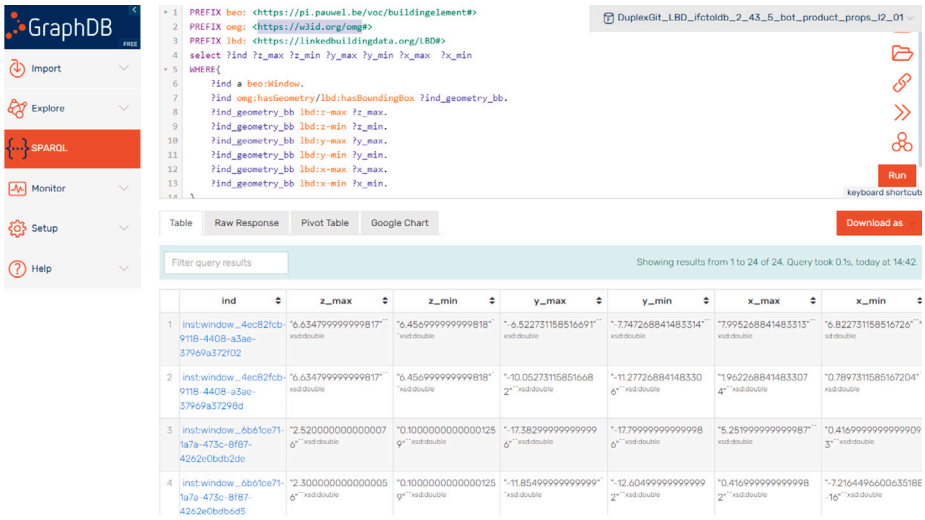


Fig. 20. IFCtoLBD V2.43.5 converter tool: geometric data of windows using GraphDB.

Table 14
Number of explicit (# Exp.) and inferred (# Inf.) RDF triples for Duplex.

#	Converter tool	Duplex					
		Virtuoso			GraphDB		
		# Exp.	# Inf.	Total	# Exp.	# Inf.	Total
1	IFC2LD V1.5 (Standard)	221,569	0	221,569	221,568	445	222,013
2	IFC2LD V1.5(Extended)	**294,730**	0	**294,730**	**294,729**	451	**295,180**
3	IFCtoRDF V0.4	227,146	0	227,146	227,145	528	227,673
4	IFCtoRDF V2.8.1	227,146	0	227,146	227,145	528	227,673
5	IFCtoLBD V2.43.5 (BOT)	4,986	7	4,993	4,986	134	5,120
6	IFCtoLBD V2.43.5 (BOT+PRODUCT)	5,251	7	5,258	5,251	134	5,385
7	IFCtoLBD V2.43.5 (BOT+ PROPS L1)	10,597	254	10,851	11,897	**632**	12,529
8	IFCtoLBD V2.43.5 (BOT+ PROPS L2)	28,014	**255**	28,269	28,014	608	28,622
9	IFCtoLBD V2.43.5 (BOT+ PROPS L3)	46,402	**255**	46,657	46,402	612	47,014
10	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L1)	10,861	254	11,115	10,861	602	11,463
11	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L2)	28,278	**255**	28,533	28,278	608	28,886
12	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L3)	46,667	**255**	46,922	46,667	612	47,279

Table 15
Number of explicit (# Exp.) and inferred (# Inf.) RDF triples for Schependomlaan.

#	Converter tool	Schependomlaan					
		Virtuoso			GraphDB		
		# Exp.	# Inf.	Total	# Exp.	# Inf.	Total
1	IFCtoRDF V0.4	**2,885,666**	0	**2,885,666**			
2	IFCtoRDF V2.8.1	2,885,661	0	2,885,661	**2,885,661**	576	**2,886,237**
3	IFCtoLBD V2.43.5 (BOT)	65,451	7	65,458	65,451	120	65,571
4	IFCtoLBD V2.43.5 (BOT+PRODUCT)	70,686	7	70,693	70,686	118	70,804
5	IFCtoLBD V2.43.5 (BOT+ PROPS L1)	209,104	840	209,944	209,104	**1,770**	210,874
6	IFCtoLBD V2.43.5 (BOT+ PROPS L2)	652,502	**841**	653,343			
7	IFCtoLBD V2.43.5 (BOT+ PROPS L3)	1,117,634	**841**	1,118,475			
8	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L1)	214,343	840	215,183	214,343	**1,770**	216,113
9	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L2)	657,741	**841**	658,582			
10	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L3)	1,122,874	**841**	1,123,715			

6.4. Interoperation with RDF tools

The following tables show the number of triples for every BIM model: Duplex in Table 14, Schependomlaan in Table 15, 7-dwelling building (IFC2X3) in Table 16, and 7-dwelling building (IFC4_ADD2) in Table 17. Five errors occurred: GraphDB could not manage five out of the ten cases for Schependomlaan dataset due to the file size being too large.[51]

The results of the SPARQL queries are summarized in Table 18, where for each tool both the dataset, the number of retrieved window individuals (#Ind.), and the time in milliseconds are shown.

[51] Free mode has a 200 MB file size limit.

Table 16
Number of explicit (# Exp.) and inferred (# Inf.) RDF triples for 7-dwelling building (IFC2X3).

#	Converter tool	7-dwelling building (IFC2X3)					
		Virtuoso			GraphDB		
		# Exp.	# Inf.	Total	# Exp.	# Inf.	Total
1	IFC2LD V1.5 (Standard)	1,073,425	0	1,073,425	1,073,424	451	1,073,875
2	IFCtoRDF V0.4	**1,120,386**	0	**1,120,386**	**1,120,386**	556	**1,120,942**
3	IFCtoRDF V2.8.1	**1,120,386**	0	**1,120,386**	**1,120,386**	556	**1,120,942**
4	IFCtoLBD V2.43.5 (BOT)	16,302	7	16,309	16,302	136	16,438
5	IFCtoLBD V2.43.5 (BOT+PRODUCT)	17,047	7	17,054	17,047	136	17,183
6	IFCtoLBD V2.43.5 (BOT+ PROPS L1)	40,978	245	41,223	40,978	572	41,550
7	IFCtoLBD V2.43.5 (BOT+ PROPS L2)	121,949	**246**	122,195	121,949	578	122,527
8	IFCtoLBD V2.43.5 (BOT+ PROPS L3)	215,704	**246**	215,950	215,704	**582**	216,286
9	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L1)	41,722	245	41,967	41,722	572	42,294
10	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L2)	122,693	**246**	122,939	122,693	578	123,271
11	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L3)	216,449	**246**	216,695	216,449	**582**	217,031

Table 17
Number of explicit (# Exp.) and inferred (# Inf.) RDF triples for 7-dwelling building (IFC4_ADD2).

#	Converter tool	7-dwelling building (IFC4_ADD2)					
		Virtuoso			GraphDB		
		# Exp.	# Inf.	Total	# Exp.	# Inf.	Total
1	IFCtoLBD V2.43.5 (BOT)	15,261	7	15,268	15,261	124	15,385
2	IFCtoLBD V2.43.5 (BOT+PRODUCT)	16,485	7	16,492	16,485	124	16,609
3	IFCtoLBD V2.43.5 (BOT+ PROPS L1)	39,668	242	39,910	39,668	568	40,236
4	IFCtoLBD V2.43.5 (BOT+ PROPS L2)	119,359	**243**	119,602	119,359	574	119,933
5	IFCtoLBD V2.43.5 (BOT+ PROPS L3)	209,852	**243**	210,095	209,852	**578**	210,430
6	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L1)	40,891	242	41,133	40,891	568	41,459
7	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L2)	120,582	**243**	120,825	120,582	574	121,156
8	IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L3)	**211,076**	**243**	**211,319**	**211,076**	**578**	**211,654**
9	KGG V0.2.0	2,808	0	2,808	2,803	92	2,895

Table 18
Results of the SPARQL queries for each tool.

Converter tool	Duplex			Schependomlaan			7-dwelling (IFC2X3)			7-dwelling (IFC4_ADD2)		
	#Ind.	Virtuoso	GraphDB	#Ind.	Virtuoso	GraphDB	#Ind.	Virtuoso	GraphDB	#Ind.	Virtuoso	GraphDB
IFC2LD V1.5 (Standard)	24	40	58				13	**0**	16			
IFC2LD V1.5 (Extended)	24	94	22									
IFCtoRDF V0.4	24	78	37	36	5	67	13	5	17			
IFCtoRDF V2.8.1	24	78	37	36	5	67	13	5	17			
IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L1)	24	**5**	**16**	36	**0**	**19**	13	**0**	**9**	13	**0**	16
IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L2)	24	42	23	0	0		13	5	10	13	10	25
IFCtoLBD V2.43.5 (BOT+PRODUCT+ PROPS L3)	24	21	24	0	0		13	5	11	13	**0**	34
KGG V0.20	24									13	52	**11**

6.5. Semantic reasoning

Firstly, it is important to note that IFCtoLBD outputs with geometric data import OMG ontology, which includes cyclic definitions where transitive properties (hasPropertyState and hasInferredPropertyState) are involved. Such combination is known to make reasoning undecidable [48]. HermiT does not support reasoning with IFCtoLBD outputs. Instead, Pellet supports reasoning but not taking into account that the properties are transitive. TrOWL supports reasoning, but it does not explain the assumed simplification. Therefore, in order to compare IFCtoLBD with the other tools for HermiT, we also considered the experiments without the geometric data (denoted "NO GEO").

Tables 19–22 shows the time (in seconds) to load and classify the ontology (Load+Classif.) and the time (in s) to check the consistency of the ontology (Consistency). Table 19 shows the reasoning times for Duplex, Table 20 for Schependomlaan, Table 21 for 7-dwelling building (IFC2x3), and Table 22 for 7-dwelling building (IFC4_ADD2).

In some cases several errors were obtained: 36 inconsistencies,[52] 16 cyclic definitions[53], 7 memory problems,[54] 3 importing problems,[55] and 7 timeouts. In particular, KGG could not be evaluated because all reasoners failed in the only supported dataset, the 7-dwelling building (IFC4_ADD2). Furthermore, the consistency time is not informative, as it is always either zero or very close to zero (0.01 s), so the discussion will focus on the loading and classification times.

[52] *"Inconsistent ontology"*.
[53] *"Cyclic dependencies"*.
[54] *"Garbage Collector (GC) overhead limit exceeded"*.
[55] *"Could not load imported ontology: ⟨https://w3id.org/props#⟩"*.

Table 19
Reasoning time (s) for Duplex BIM model.

Converter tool	Duplex					
	HermiT		TrOWL		Pellet	
	Load+Classif.	Consistency	Load+Classif.	Consistency	Load+Classif.	Consistency
IFC2LD V1.5 (Standard)	Inconsistent ontology		31.35	0	Inconsistent ontology	
IFC2LD V1.5 (Extended)	Inconsistent ontology		177.15	0	Inconsistent ontology	
IFCtoRDF V0.4	343.37	0	GC overhead limit exceeded		27.29	0
IFCtoRDF V2.8.1	322.87	0	GC overhead limit exceeded		20.71	0
IFCtoLBD V2.43.5 (BOT)	Cyclic dependency		2.35	0	2.12	0
IFCtoLBD V2.43.5 (BOT+PRODUCT)	Cyclic dependency		2.40	0	2.10	0
IFCtoLBD V2.43.5 (BOT+PROPS L1)	Cyclic dependency		6.50	0	2.46	0
IFCtoLBD V2.43.5 (BOT+PROPS L2)	Cyclic dependency		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (BOT+PROPS L3)	Cyclic dependency		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	Cyclic dependency		3.16	0	2.52	0
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	Cyclic dependency		5.38	0	3.00	0
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	Cyclic dependency		12.72	0	3.11	0
IFCtoLBD V2.43.5 (NO GEO+BOT)	**1.41**	0	**1.16**	0	**0.98**	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT)	1.49	0	1.44	0	1.42	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L1)	1.80	0	4.20	0	1.64	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L2)	Inconsistent ontology		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L3)	Inconsistent ontology		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L1)	1.99	0	2.03	3.33E−4	1.71	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L2)	1.70	0	2.27	0	1.54	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L3)	1.93	0	5.82	0	1.84	0

Table 20
Reasoning time (s) for Schependomlaan BIM model.

Converter tool	Schependomlaan					
	HermiT		TrOWL		Pellet	
	Load+Classif.	Consistency	Load+Classif.	Consistency	Load+Classif.	Consistency
IFCtoRDF V0.4	Timeout		GC overhead limit exceeded		318.18	0
IFCtoRDF V2.8.1	Timeout		GC overhead limit exceeded		367.59	0
IFCtoLBD V2.43.5 (BOT)	Cyclic dependency		14.63	0	4.20	0
IFCtoLBD V2.43.5 (BOT+PRODUCT)	Cyclic dependency		13.02	0	4.08	0
IFCtoLBD V2.43.5 (BOT+PROPS L1)	Cyclic dependency		147.58	0	6.175	0
IFCtoLBD V2.43.5 (BOT+PROPS L2)	Cyclic dependency		2,483.62	0	15.65	0
IFCtoLBD V2.43.5 (BOT+PROPS L3)	Cyclic dependency		Timeout		25.67	0
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	Cyclic dependency		47.63	0	9.39	0
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	Cyclic dependency		Timeout		13.83	0
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	Cyclic dependency		Timeout		34.79	0
IFCtoLBD V2.43.5 (NO GEO+BOT)	**1.68**	0	**1.50**	0	**1.27**	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT)	1.77	0	1.92	0	1.68	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L1)	3.20	0	78.30	0	3.87	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L2)	9.73	0	Timeout		13.23	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L3)	18.25	0	Timeout		20.56	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L1)	3.45	0	66.70	0	3.83	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L2)	7.92	0	Timeout		9.53	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L3)	17.68	0	Timeout		20.31	0

7. Discussion

This section discusses the main findings of our experiments, based on the results reported in Section 6. In the next subsections, each research question will be analyzed in detail: Section 7.1 discusses conversion time and robustness (RQ1), Section 7.2 discusses our findings regarding the ontology metrics (RQ2), Section 7.3 discusses the preservation of information (RQ3), Section 7.4 discusses our experiments with RDF tools (RQ4), and Section 7.5 discusses our experiments regarding OWL reasoners (RQ5). As a summary, Table 23 shows the most appropriate converter tool according to different criteria

7.1. Conversion time and robustness

This section discusses the results reported in Table 5. From the point of view of the datasets, Duplex was the dataset with more successful conversions (12 out of 14), followed by 7-dwelling building in IFC2X3 (11 conversions), Schependomlaan (10), and 7-dwelling building in IFC4 (9). This makes a total of 42 correct conversions which, multiplied by 3 runs, gives 126 output files.

From the point of view of the tools, IFCtoLBD was the most robust tool for finishing the conversion. Moreover, KGG was the fastest tool in the single case where it finished (7-dwelling in IFC4), and IFCtoRDF was the fastest one in two out of the three cases where it finished (Schependomlaan

Table 21
Reasoning time (s) for 7-dwelling building (IFC2X3) BIM model.

Converter tool	7-dwelling building (IFC2X3)					
	HermiT		TrOWL		Pellet	
	Load+Classif.	Consistency	Load+Classif.	Consistency	Load+Classif.	Consistency
IFC2LD V1.5 (Standard)	Inconsistent ontology		GC overhead limit exceeded		Inconsistent ontology	
IFCtoRDF V0.4	6,678.46	0	GC overhead limit exceeded		77.93	0
IFCtoRDF V2.8.1	6,295.07	0	GC overhead limit exceeded		80.14	0
IFCtoLBD V2.43.5 (BOT)	Cyclic dependency		3.60	3.33E−4	2.85	0
IFCtoLBD V2.43.5 (BOT+PRODUCT)	Cyclic dependency		3.60	0	2.85	0
IFCtoLBD V2.43.5 (BOT+PROPS L1)	Cyclic dependency		21.2	0	3.27	0
IFCtoLBD V2.43.5 (BOT+PROPS L2)	Cyclic dependency		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (BOT+PROPS L3)	Cyclic dependency		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	Cyclic dependency		5.10	0	4.23	0
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	Cyclic dependency		50.69	0	4.16	0
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	Cyclic dependency		485.94	0	6.15	0
IFCtoLBD V2.43.5 (NO GEO+BOT)	**1.27**	0	**0.98**	0	**0.92**	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT)	1.47	0	1.39	0	1.47	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L1)	1.98	0	12.33	0	2.00	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L2)	Inconsistent ontology		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L3)	Inconsistent ontology		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L1)	2.54	0	2.76	0	2.162	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L2)	3.20	0	18.73	0	2.73	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L3)	4.24	0	228,54	0	4.47	0

Table 22
Reasoning time (s) for the 7-dwelling building (IFC4_ADD2) BIM model.

Converter tool	7-dwelling building (IFC4_ADD2)					
	HermiT		TrOWL		Pellet	
	Load+Classif.	Consistency	Load+Classif.	Consistency	Load+Classif.	Consistency
IFCtoLBD V2.43.5 (BOT)	Cyclic dependency		5.20	0	3.53	3.33E−4
IFCtoLBD V2.43.5 (BOT+PRODUCT)	Cyclic dependency		4.26	0	3.82	0
IFCtoLBD V2.43.5 (BOT+PROPS L1)	Cyclic dependency		23.77	6.66E−4	4.91	0
IFCtoLBD V2.43.5 (BOT+PROPS L2)	Cyclic dependency		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (BOT+PROPS L3)	Cyclic dependency		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	Cyclic dependency		6.90	0	5.48	0
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)	Cyclic dependency		50.62	3.33E−4	5.76	3.33E−4
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)	Cyclic dependency		410.22	0	7.76	0
IFCtoLBD V2.43.5 (NO GEO+BOT)	**1.43**	0	**1.11**	0	**0.98**	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT)	1.62	0	1.34	0	1.33	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L1)	3.70	0	13.00	0	2.14	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L2)	Inconsistent ontology		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (NO GEO+BOT+PROPS L3)	Inconsistent ontology		Inconsistent ontology		Inconsistent ontology	
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L1)	2.38	0	2.50	0	1.83	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L2)	2.80	0	18.40	0	2.44	0
IFCtoLBD V2.43.5 (NO GEO+BOT+PRODUCT+PROPS L3)	3.44	0	130.56	0	4.10	0
KGG V0.2.0	Could not load imported ontology: ⟨https://w3id.org/props#⟩					

and 7-dwelling in IFC2X3). In the other dataset, IFCtoLD was the fastest one. Furthermore, there are significant differences between the fastest and the slowest tool, ranging from 20 (for 7-dwelling in IFC2X3) to 99 s (for Schependomlaan).

7.2. Output metrics

It is essential to have well-defined ontologies that define a reference schema for the converter data. However, different tools supported different ontologies, and different variants of ifcOWL ontology, as shown in Tables 1 and 6.

Regarding the Description Logic expressivity of the outputs, Table 11 shows that KGG and IFCtoLBD models are the most expressive ones. While KGG supports nominal concepts ($\mathcal{O}$), IFCtoLBD supports functional properties ($\mathcal{F}$).

Now, let us discuss the results reported in Tables 6–10.

As expected, IFC2LD V1.5 has more elements in the *Extended* than in the *Standard* configuration. In particular, it has more classes, object properties, total axioms, class assertions, and object property assertions. For example, there is a difference of 52 classes in the Duplex BIM model.

In Schependomlaan, the different versions (V0.4 and V2.8.1) of IFCtoRDF generate outputs with a different number of RDF triples and different ontology metrics. In the 7-dwelling building (IFC2X3), both the number of triples and the metrics coincide.

In the 7-dwelling building, IFCtoLBD V2.4 has similar but different metrics in the IFC2X3 and the IFC4_ADD2 versions.

IFCtoRDF models have the largest number of classes, individuals, and class assertions in the three datasets where it finished correctly, whereas IFC2LD models have the largest number of data properties and data property assertions in the two datasets where it finished correctly. In the

Table 23
Comparison of converter tools.

		IFC2LD	IFCtoRDF	IFCtoLBD	KGG
Scope	General use	✓	✓	✓	
	Internet of Things				✓
IFC version	IFC2 × 3	✓	✓	✓	
	IFC4			✓	✓
Conversion	Conversion speed (IFC2 × 3)		✓		
	Conversion speed (IFC4)				✓
	Robustness (errors in conversion)	✓			
Output	Conversion size	✓			
Preservation	Preservation of classes		✓		
	Preservation of object properties	✓			
	Preservation of data properties			✓	
	Preservation of individuals		✓		
	Preservation of geometric data	✓	✓		
Graph querying	Querying using SPARQL				✓
Ontology reasoning	Querying with semantic reasoner			✓	

Table 24
Comparison of the property types used by different tools.

Tool	Data	Object	Annotation	Anonymous
IFC2LD V1.5	✓	✓		✓
IFCtoRDF V0.4 and V2.8.1			✓	
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L1)	✓		✓	
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L2)		✓	✓	
IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS L3)		✓	✓	
KGG V0.2.0			✓	

7-dwelling building (IFC4_ADD2), where neither IFCtoRDF nor IFC2LD finished correctly, KGG model has more classes (1032, because the BRICK ontology contains 837 classes) than IFCtoLBD models, but IFCtoLBD wins in the other ontology metrics.

Regarding object property assertions, the largest number happens for IFC2LD with the Extended configuration, but it only finishes in Duplex dataset. In the other datasets, the winner was IFCtoRDF when it finished.

IFCtoLBD models have the largest number of object properties. As expected, Level $L3$ models have more object properties, as they are used to navigate through the levels, but IFCtoLBD still has more object properties when using Level $L1$. Furthermore, in Level $L3$, the number of individuals, axioms and class assertions is higher than in the rest of levels. On the contrary, in $L1$ there are more datatype properties. Unsurprisingly, the more imported ontologies, the larger number of classes, properties, individuals, and axioms: the largest numbers happen for BOT+PRODUCT+PROPS and the smallest ones for BOT.

7.3. Preserving information

Regarding the preservation of classes, Table 12 shows that IFC Java viewer does not provide a realistic way to compare with the original models, as the number of classes is smaller.

Regarding the preservation of individuals, the situation is the same, as illustrated in Table 13. While IFC Java viewer, IFC2LD and IFCtoRDF produced the same number of individuals in Duplex, IFCtoLBD does not coincide with them. Furthermore, IFC Java viewer, IFC2LD and IFCtoRDF do not coincide in the other datasets. For example, in 7-dwelling (IFC2X3) IFC2LD and IFCtoRDF added a second individual to the class IfcSite, which only had one individual in the original model. Note that IFCtoLBD sometimes groups in a superclass all the instances of its subclasses. For example, in Duplex, IFCtoLBD includes in IfcElement all the instances of its subclasses in the original model, so general classes can have more individuals in the converted files.

Regarding the preservation of properties, different tools offer different solutions. Table 24 summarizes our results regarding the preservation of property types, showing for each converter tool if it uses data properties, object properties, annotation properties, or anonymous individuals. Ensuring that the resulting files are accepted by current ontology editors or ontology reasoners requires further attention. In particular, we noticed that some properties that were not explicitly defined in the ontology schemes were interpreted as annotation properties by the ontology editor Protégé. Therefore, if one wants to reason with the output files by submitting queries to a semantic reasoner using Protégé graphical interface, such annotations are not considered. Instead, the user has to use an RDF graph software or the OWL API. This happened in all converters except IFC2LD. However, IFC2LD requires blank nodes, which cannot be adequately managed from Protégé or the OWL API methods that interact with a semantic reasoner.

Regarding the management of geometric data, IFC2LD, IFCtoRDF, and IFCtoLBD can keep them but, contrary to CDE, in a Semantic Web language. IFCtoLBD can optionally ignore such data, and KGG always does, reducing the size of the converted files and claiming that this is more appropriate for a specific application type. For example, KGG is mainly thought for Industry 4.0 and the Internet of Things, where filesize is critical. Furthermore, both IFC2LD and IFCtoRDF use different ways to represent the geometric data, so they require different SPARQL queries to access the information.

7.4. Interoperation with RDF tools

Let us firstly discuss the results in Tables 14–17.

The number of triples is impressively different across converter tools. For example, in the Schependomlaan dataset, IFCtoRDF obtained more than 2.88 million RDF triples but IFCtoLBD V2.43.5 obtained less than 66,000 in one of the configurations (BOT). Furthermore, different configurations or versions of the same tool lead to a different number of triples. For example, in Schependomlaan, IFCtoRDF V0.4 and IFCtoRDF V2.8.1 generate outputs with a different number of RDF triples.

Usually, both graph databases have the same number of explicit triples (i.e., both of them load the same number of triples from the input RDF file), as expected. Curiously, there are 7 exceptions where Virtuoso shows more triples (in 5 cases, the difference is just one triple). Because of this difference, we also checked the number of explicit triples using Stardog V8.0.1 and it coincided with GraphDB.

The numbers of inferred triples are very different, as they depend on the reasoner used: the number of inferred triples is always larger using GraphDB. Apparently, the main reason why Virtuoso provides less triples is the less sophisticated inference rules. Appendix D shows detailed examples of the different inferences.

Another important finding is that none of the tools provides the highest number of explicit triples in all the cases: IFC2LD does in Duplex, IFCtoRDF does in Scherpendomlaan and 7-dwelling building (IFC2X3), while IFCtoLBD does it in the 7-dwelling building in IFC4.[56] Regarding inferred triples, IFC2LD models had the highest number in all datasets.[57]

In IFCtoLBD, the general trend is that the more ontologies selected, the higher the number of explicit triples, but there is an exception. In Duplex, using BOT+PRODUCT+PROPS $L1$ and GraphDB leads to a smaller number of triples than using BOT+PROPS $L1$,[58] and this happens for both explicit and implicit triples.

Now, let us discuss the results in Table 18. We can see that Virtuoso is faster, which is expected as it handles a smaller number of inferred triples.

For each dataset, all tools correctly retrieved the same windows,[59] although the answering times were different. For example, the fastest one (IFCtoLBD) was about 18 times faster than the slowest one (IFC2LD) in the Duplex dataset.

In general, IFCtoLBD models were the fastest ones, both using Virtuoso and GraphDB, which is expected because of the smaller number of RDF triples. There is one exception: KGG was the fastest model in the only dataset it supported (7-dwelling IFC4) using GraphDB; although IFCtoLBD model was still faster using Virtuoso.

7.5. Semantic reasoning

This section discusses the results in Tables 19–22. The fastest reasoning times were always obtained for IFCtoLBD (in particular, for the BOT configuration, with the simplest ontology schema). In such cases, Pellet was the faster reasoner on all datasets, followed by TrOWL. Differences between converter tools can be significant: in the 7-dwelling building (IFC2X3), the IFCtoLBD V2.43.5 (BOT) model is more than 100 min faster than the IFCtoRDF models. In Duplex, there is a difference of more than 5 min for the same case.

Now, let us discuss the configurations of the three converter tools for which there are data. To compare IFC2LD V1.5 Standard and Extended configurations, only data for the TrOWL reasoner in Duplex are available. As expected, reasoning time with the Standard configuration is clearly (more than 5 times) smaller.

In IFCtoRDF versions, both HermiT and Pellet reasoners were tested without errors in Duplex and 7-dwelling building (IFC2X3), while only Pellet finished in Schependomlaan. For those three cases, Pellet was the quickest. TrOWL returned a memory error in the three datasets.

In IFCtoLBD V2.43.5, none of the reasoners finished in the cases BOT+PROPS $L2$ and $L3$ in Duplex and the two 7-dwelling buildings. In Schependomlaan, TrOWL reached the timeout in about half of the cases. Finally, reasoning with geometric information took about twice running time is TrOWL and Pellet and is not allowed on HermiT.

8. Conclusions and future work

This paper reports the results of an empirical analysis of different tools (IFC2LD, IFCtoRDF, IFCtoLBD, and KGG) that convert Building Information Model (BIM) files in the standard Industry Foundation Classes (IFC) format into a semantic format, namely RDF triples, possibly concerning a reference OWL ontology. Such output formats are fundamental to enhancing applications' reuse, maintenance, correctness, and interpretability using BIM files.

Four real BIM models were considered in our experiments. The models were translated with different converter tools and evaluated using different criteria, including the time (to convert the model, to answer SPARQL queries, and to perform semantic reasoning) and the elements in the output files (number and types of elements). Important differences between the converter tools were observed. For example, to solve SPARQL queries, we noticed that computation times were four times larger in some cases, and to perform semantic reasoning, the difference could be of more than 100 min.

To summarize our results, KGG is faster at converting and querying using SPARQL but requires IFC4 input files and provides small output files, so it is only suitable for specific applications where file size is a critical limitation. If geometric data are important, IFCtoRDF, IFC2LD, and IFCtoLBD are the options.

IFCtoLBD generates more triples and preserves more object properties than other tools, is the most reliable at converting (giving errors less frequently), and is the most appropriate tool to reason using a semantic reasoner (but only without geometric data). IFCtoRDF is faster at converting IFC models and is better at preserving classes and individuals. On the other hand, IFC2LD preserves more data properties.

On the one hand, our results will be of relevance for the users that must choose the most appropriate converter for their needs. On the other hand, our results will help developers to improve future versions of their tools by correcting some (or hopefully all) of the identified limitations.

[56] This is observed in Table 14 row 2, Table 15 row 1, Table 16 rows 2–3, and Table 17 row 8.

[57] See the bold values in columns "# Inf". at Tables 14–17.

[58] See Table 14 rows 7 and 10.

[59] In Schependomlaan, the output files did not include the property values in two cases ($L2$ and $L3$), so no windows were retrieved.

Our study has some limitations: it is restricted to four tools and four datasets which only cover IFC models up to IFC4 version. Therefore, there are many exciting ideas for future work. Firstly, it would be interesting to consider more datasets (using IFC4) supported by KGG. Furthermore, future versions of the tools could be analyzed. In particular, converter tools should be adapted to support the latest version of IFC (IFC5). Then, it would be interesting to see the behavior of the tools when converting such BIM models. Last but not least, it would be interesting to evaluate other converter tools even if they do not start from an input IFC model. For example, evaluating the BHoM RDF converter tool [49] with a real-world object model could be valuable.

CRediT authorship contribution statement

Ignacio Huitzil: Writing – original draft, Visualization, Validation, Software, Investigation, Formal analysis, Data curation. **Miguel Molina-Solana:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Juan Gómez-Romero:** Writing – review & editing, Funding acquisition, Conceptualization. **Marco Schorlemmer:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Conceptualization. **Pere Garcia-Calvés:** Writing – review & editing, Conceptualization. **Nardine Osman:** Writing – review & editing, Conceptualization. **Josep Coll:** Writing – review & editing, Resources, Funding acquisition. **Fernando Bobillo:** Writing – original draft, Visualization, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially funded by Imperial College's Centre for Systems Engineering and Innovation under its 2019 call for small projects (project ♯2). I. Huitzil was partially supported by the Italian MUR under the PRIN project PINPOINT Prot. 2020FNEB27, CUP H45E21000210001. M. Molina-Solana and J. Gómez-Romero were partially funded by the Spanish Ministry of Science (SINERGY project, PID2021-125537NA-I00) and the NextGenerationEU funds (IA4TES project, MIA.2021.M04.0008). I. Huitzil, M. Schorlemmer, P. Garcia-Calvés, N. Osman, and J. Coll were partially supported by the Government of Catalonia under ACCIÓ project *smartNorms4BIM* (INNOTECRD19-1-0045) and AGAUR project *smartNorms* (2021 PROD 00031). F. Bobillo and I. Huitzil were partially supported by the I+D+i project PID2020-113903RB-I00, funded by MCIN/AEI/10.13039/501100011033, and the project T42_23R, funded by Gobierno de Aragón. We are also grateful to Carlos Bobed for some technical help with Virtuoso configuration and to the anonymous reviewers for their comments.

Appendix A. Turtle syntax

In a Turtle file, triples are separated using the symbol "`.`" in general. Prefixes are defined in the form `@prefix <prefixName> <definition>`. Concept assertions are represented by linking an individual with its class via the property `rdf:type`, which is usually abbreviated as `a`. Subclass axioms are represented using the property `rdfs:subClassOf`.

Example 2. Let us represent some knowledge using Turtle syntax: Window01 is a member of IfcWindow class, Window01 is made of a type of aluminum (2000 series), Window01 is made of oak (a type of wood), the height of Window01 is 1634 (mm), BasicWindow is a subclass of IfcWindow, and AluminumWindow is any window made of aluminum material.

```
@prefix ifc: <https://standards.buildingsmart.org/IFC/DEV/IFC2x3/TC1/OWL#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
ifc:Window01       rdf:type ifc:IfcWindow .
ifc:Window01       ifc:madeOf ifc:ALUMINUM2000 .
ifc:Window01       ifc:madeOf ifc:OAK .
ifc:Window01       ifc:overallHeight "1634"^^xsd:integer .
ifc:BasicWindow    rdfs:subclassOf ifc:IfcWindow .
ifc:AluminumWindow owl:equivalentClass
     [ owl:intersectionOf ( ifc:IfcWindow
                            [ a owl:Restriction ;
                              owl:onProperty ifc:madeOf ;
                              owl:someValuesFrom ifc:ALUMINUM
                            ]
                          )
     ] .
```

Note that the definition of ifc:AluminumWindow requires some OWL constructors (such as `owl:equivalentClass`, `owl:intersectionOf` or `owl:someValuesFrom`) whose semantics cannot be represented in RDF.

Turtle syntax has some optimizations to reduce the size of the files. For example, when several triples share the same subject, they are separated using the symbol "`;`" and the subject is only written before the first triple. Similarly, when several triples share the same subject and property, they are separated using the symbol "`,`" and the subject and the property are only written before the first triple.

Example 3. Let us represent again Example 2 using a more compact Turtle syntax:

```
@prefix ifc: <https://standards.buildingsmart.org/IFC/DEV/IFC2x3/TC1/OWL#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
ifc:Window01       a ifc:IfcWindow ;
                   ifc:madeOf ifc:ALUMINUM2000,
                              ifc:OAK ;
                   ifc:overallHeight "1634"^^xsd:integer .
ifc:BasicWindow    rdfs:subclassOf ifc:IfcWindow .
ifc:AluminumWindow owl:equivalentClass
     [ owl:intersectionOf ( ifc:IfcWindow
                            [ a owl:Restriction ;
                              owl:onProperty ifc:madeOf ;
                              owl:someValuesFrom ifc:ALUMINUM
                            ]
                          )
     ] .
```

Appendix B. Examples of RDF triples using different approaches to represent properties

Example 4. In the output file computed using IFC2LD V1.5, the relevant RDF triples are the following ones:

```
model:GUID_1hOSvn6df7F8_7GcBWlRLx
   a   ifc:IfcWindow ;
   ifc:overallHeight    "2.199999999999996"^^xsd:double .
```

The ifcOWL (Standard) definition of overallHeight shows that it is indeed a data property:

```
ifc:overallHeight  a  owl:FunctionalProperty , owl:DatatypeProperty .
```

Example 5. In the output of IFCtoRDF V2.8.1, the relevant RDF triples are:

```
<http://sid.cps.unizar.es/converters.owl/GUID/
6b61ce71-1a7a-473c-8f87-4262e0bdb57b>
    a   ifcowl:IfcWindow ;
    ifcowl:overallHeight_IfcWindow  inst:IfcPositiveLengthMeasure_46591 .

inst:IfcPositiveLengthMeasure_46591
   a   ifcowl:IfcPositiveLengthMeasure ;
   express:hasDouble  "2.199999999999996"^^xsd:double .
```

The ifcOWL schema definition of overallHeight_IfcWindow is:

```
ifc:overallHeight_IfcWindow
    a   owl:ObjectProperty , owl:FunctionalProperty ;
    rdfs:domain  ifc:IfcWindow ;
    rdfs:label   "OverallHeight" ;
    rdfs:range   ifc:IfcPositiveLengthMeasure .
```

Example 6. In the output of IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS *L*1), the relevant RDF triples for these properties are:

```
inst:window_6b61ce71-1a7a-473c-8f87-4262e0bdb57b
   rdf:type   beo:Window ;
   rdf:type   bot:Element ;
   props:height_property_simple    2.2 ;
   props:overallHeightIfcWindow_attribute_simple
        "2.199999999999996"^^<http://www.w3.org/2001/XMLSchema#double> ;
```

The definition of height in the PROPS ontology schema is:

```
<http://lbd.arch.rwth-aachen.de/props#height>
   rdf:type owl:ObjectProperty ;
   rdfs:subPropertyOf <http://lbd.arch.rwth-aachen.de/props#property> ;
   rdfs:domain <https://w3id.org/bot#Element> ;
   rdfs:range <https://w3id.org/opm#Property> .
```

The definition of height_simple in the PROPS ontology schema is:

```
<http://lbd.arch.rwth-aachen.de/props#height_simple>
   rdf:type owl:ObjectProperty ;
   rdfs:domain <https://w3id.org/bot#Element> .
```

Example 7. In the output given by IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS *L*2), some relevant RDF triples are:

```
inst:window_6b61ce71-1a7a-473c-8f87-4262e0bdb57b
   rdf:type     beo:Window ;
   rdf:type     bot:Element ;
   props:overallHeightIfcWindow
                inst:overallHeightIfcWindow_6b61ce71-1a7a-473c-8f87-4262e0bdb57b ;
   props:height inst:height_6b61ce71-1a7a-473c-8f87-4262e0bdb57b .
```

```
inst:overallHeightIfcWindow_6b61ce71-1a7a-473c-8f87-4262e0bdb57b
   rdf:type      <https://w3id.org/opm#Property> ;
   schema:value  "2.199999999999996"^^<http://www.w3.org/2001/XMLSchema#double> ;
   smls:unit     unit:MilliM .

inst:height_6b61ce71-1a7a-473c-8f87-4262e0bdb57b
   rdf:type      <https://w3id.org/opm#Property> ;
   rdfs:label    "PSet_Revit_Type_Dimensions:height" ;
   schema:value  2.2 .
```

The definition of height in the PROPS ontology schema is:

```
<http://lbd.arch.rwth-aachen.de/props#height>
   rdf:type owl:ObjectProperty ;
   rdfs:subPropertyOf <http://lbd.arch.rwth-aachen.de/props#property> ;
   rdfs:domain <https://w3id.org/bot#Element> ;
   rdfs:range <https://w3id.org/opm#Property> .
```

Example 8. In the output of IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS $L3$), the relevant RDF triples are:

```
inst:window_6b61ce71-1a7a-473c-8f87-4262e0bdb57b
   rdf:type   beo:Window ;
   rdf:type   bot:Element ;
   props:height   inst:height_6b61ce71-1a7a-473c-8f87-4262e0bdb57b .

inst:height_6b61ce71-1a7a-473c-8f87-4262e0bdb57b
   rdf:type   opm:Property ;
   rdfs:label "PSet_Revit_Type_Dimensions:height" ;
   opm:hasPropertyState
        inst:state_height_6b61ce71-1a7a-473c-8f87-4262e0bdb57b_p3629 .

inst:state_height_6b61ce71-1a7a-473c-8f87-4262e0bdb57b_p3629
   rdf:type   opm:CurrentPropertyState ;
   prov:generatedAtTime  "2024-09-27T22:46:33.5592027" ;
   schema:value 2.2 .
```

Example 9. In the output of KGG V0.2.0, some relevant RDF triples are:

```
om:Element_133698 a  beo:Window , bot:Element ;
   rdfs:label       "Fusteria batent 1F + Fixe 1F:Finestra
   _Batent_1F_Fixe_1F-Metalica-1200x1200:682253" ;
   props:hasArea        "3.27460400000012"^^xsd:double .
```

Appendix C. SPARQL queries

Let us start with the queries used in the Duplex dataset. In the BIM model converted with IFC2LD V1.5 (Standard), overallHeight and overallWidth are data properties that directly link a real value. The SPARQL query is:

```
SELECT ?ind ?height ?width
WHERE {
   ?ind a ifc:IfcWindow.
   ?ind ifc:overallHeight ?height.
   ?ind ifc:overallWidth ?width.
}
```

In the BIM model converted with IFCtoRDF V2.81, overallHeight_IfcWindow and overallWidth_IfcWindow are object properties that directly relate an individual ?ind to an individual ?Hind that is linked to a real value. The SPARQL query is:

```
SELECT ?ind ?height ?width
WHERE {
  ?ind  a ifcowl:IfcWindow.
  ?ind  ifcowl:overallHeight_IfcWindow ?Hind.
  ?Hind express:hasDouble ?height.
  ?ind  ifcowl:overallWidth_IfcWindow ?Wind.
  ?Wind express:hasDouble ?width.
}
```

In the BIM model converted with IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS $L1$), it is possible to use two different data properties defined on PROPS schema for the height, namely props:height_property_simple and props:overallHeightIfcWindow_attribute_simple, and the result has a variation of one hundredth. For example, for the individual in Fig. 9, ?height1 = 2.2 but ?height2 = 2.19999... The same problem happens with the width. The SPARQL query is:

```
SELECT ?ind ?height1 ?width1 ?height2 ?width2
WHERE {
  ?ind a beo:Window.
  ?ind props:height_property_simple ?height1.
  ?ind props:width_property_simple  ?width1.
  ?ind props:overallHeightIfcWindow_attribute_simple  ?height2.
  ?ind props:overallWidthIfcWindow_attribute_simple ?width2.
}
```

In the BIM model converted with IFCtoLBD V2.43.5 (BOT+ PRODUCT+PROPS *L2*), height and width are object properties that directly associate an individual to another individual that links a real value. The SPARQL query is:

```
SELECT ?ind ?height ?width
WHERE {
  ?ind  a beo:Window.
  ?ind  props:height ?hind.
  ?hind schema:value ?height.
  ?ind  props:width ?wind.
  ?wind schema:value ?width.
}
```

In the BIM model converted with IFCtoLBD V2.43.5 (BOT+PRODUCT+PROPS *L3*), height and width object properties also have a path of length two to arrive to the real values. The SPARQL query is:

```
SELECT ?ind ?height ?width
WHERE {
  ?ind     a beo:Window.
  ?ind     props:height ?hInd.
  ?hInd    opm:hasPropertyState ?hasInd1.
  ?hasInd1 schema:value  ?height.
  ?ind     props:width ?wInd.
  ?wInd    opm:hasPropertyState ?hasInd2.
  ?hasInd2 schema:value ?width.
}
```

In Schependomlaan and 7-dwelling (IFC2X3), the queries are similar but, in cases with two similar properties (e.g., props:height_property_simple and overallHeightIfcWindow_attribute_simple), only one (props:height_property_simple) is queried.

Finally, let us show the queries used in the BIM model converted with KGG V0.2.0. Here, the area was queried using the data property (hasArea). The SPARQL query is:

```
SELECT ?ind ?area
WHERE{
  ?ind a beo:Window.
  ?ind props:hasArea ?area.
}
```

To compare KGG with at least one converter tool, the queries in IFCtoLBD V2.43.5 also retrieve the area. In particular, the queries are similar to those used in the Duplex dataset but, rather than querying the height and width, the area was queried using properties area_property_simple (in Level *L1*) and area (in Levels *L2* and *L3*).

Appendix D. Examples of inferred triples

To illustrate the differences between Virtuoso and GraphDB to discover inferred triples, some examples will be provided. Both of them added new triples stating that the properties using lbd and props prefixes are indeed properties, such as:

```
lbd:hasBoundingBox rdf:type rdf:Property .
props:height_property_simple rdf:type rdf:Property .
```

GraphDB additionally added triples stating subproperty relationships and regarding properties using other prefixes, such as:

```
bot:hasBuilding rdf:type rdf:Property .
lbd:hasBoundingBox rdfs:subPropertyOf lbd:hasBoundingBox .
omg:hasGeometry rdf:type rdf:Property .
owl:differentFrom rdf:type owl:SymmetricProperty .
props:height_property_simple rdfs:subPropertyOf props:height_property_simple .
proton:transitiveOver rdf:type rdf:Property .
rdf:type rdf:type rdf:Property .
rdfs:subClassOf rdf:type owl:TransitiveProperty .
xsd:string rdf:type rdfs:Class .
```

Data availability

Data will be made available on request.

References

[1] R. Teulier, M. Bagieu, Building Information Modeling: Shared Modeling, Mutual Data, the New Art of Building, Wiley, 2023, http://dx.doi.org/10.1002/9781394264933.
[2] A. Borrmann, M. König, C. Koch, J. Beetz, Building information modeling: Why? what? how? in: Building Information Modeling: Technology Foundations and Industry Practice, Springer International Publishing, 2018, pp. 1–24, http://dx.doi.org/10.1007/978-3-319-92862-3_1.
[3] A. Sawhney, M. Riley, J. Irizarry, Construction 4.0: an Innovation Platform for the Built Environment, Routledge, 2020, http://dx.doi.org/10.1201/9780429398100.
[4] M. Poljansek, Building Information Modelling (BIM) Standardization, Tech. Rep. JRC109656, European Joint Research Centre (JRC), 2017, http://dx.doi.org/10.2760/36471.
[5] buildingSMART International, Industry foundation classes (IFC), 2024, https://technical.buildingsmart.org/standards/ifc/.
[6] K. Farghaly, R. Soman, A.S. Zhou, The evolution of ontology in AEC: A two-decade synthesis, application domains, and future directions, J. Ind. Inf. Integr. 36 (2023) 100519, http://dx.doi.org/10.1016/j.jii.2023.100519.
[7] B. Cuenca-Grau, I. Horrocks, B. Motik, B. Parsia, P.F. Patel-Schneider, U. Sattler, OWL 2: The next step for OWL, J. Web Semant. 6 (4) (2008) 309–322, http://dx.doi.org/10.1016/j.websem.2008.05.001.
[8] W3C OWL Working Group, OWL 2 overview, 2012, http://www.w3.org/TR/owl2-overview/.
[9] W3C, RDF primer, 2004, http://www.w3.org/TR/rdf-primer.

[10] S. Staab, R. Studer (Eds.), Handbook on Ontologies, International Handbooks on Information Systems, Springer, 2004.
[11] J. Beetz, J. van Leewen, B. de Vries, IfcOWL: A case of transforming EXPRESS schemas into ontologies, Artif. Intell. Eng. Des. Anal. Manuf. 23 (2009) 89–101, http://dx.doi.org/10.1017/S0890060409000122.
[12] P. Pauwels, S. Zhang, Y.-C. Lee, Semantic web technologies in AEC industry: A literature overview, Autom. Constr. 73 (2017) 145–165, http://dx.doi.org/10.1016/j.autcon.2016.10.003.
[13] N. Luo, M. Pritoni, T. Hong, An overview of data tools for representing and managing building information and performance data, Renew. Sustain. Energy Rev. 147 (2021) 111224, http://dx.doi.org/10.1016/j.rser.2021.111224.
[14] R.K. Soman, M. Molina-Solana, J.K. Whyte, Linked-Data based Constraint-Checking (LDCC) to support look-ahead planning in construction, Autom. Constr. 120 (2020) 103369, http://dx.doi.org/10.1016/j.autcon.2020.103369.
[15] I. Huitzil, M. Schorlemmer, N. Osman, P. Garcia, J. Coll, X. Coll, Towards automated compliance checking of building regulations: smartNorms4BIM, in: Artificial Intelligence Research and Development — Proceedings of the 24th International Conference of the Catalan Association for Artificial Intelligence, CCIA 2022, in: Frontiers in Artificial Intelligence and Applications, vol. 356, IOS Press, 2022, pp. 95–104, http://dx.doi.org/10.3233/FAIA220322.
[16] M. Fahad, N. Bus, B. Fies, Semantic BIM reasoner for the verification of IFC models, in: Proceedings of the 12th European Conference on Product and Process Modelling, ECPPM 2018, CRC Press, 2018, pp. 361–368, http://dx.doi.org/10.1201/9780429506215-45.
[17] J. Gómez-Romero, F. Bobillo, M. Ros, M. Molina-Solana, M.D. Ruiz, M.J. Martín-Bautista, A fuzzy extension of the semantic building information model, Autom. Constr. 57 (2015) 202–212, http://dx.doi.org/10.1016/j.autcon.2015.04.007.
[18] I. Huitzil, M. Molina-Solana, J. Gómez-Romero, F. Bobillo, Minimalistic fuzzy ontology reasoning: An application to building information modeling, Appl. Soft Comput. 103 (2021) 107158, http://dx.doi.org/10.1016/j.asoc.2021.107158.
[19] P. Pauwels, W. Terkaj, EXPRESS to OWL for construction industry: Towards a recommendable and usable ifcOWL ontology, Autom. Constr. 63 (2016) 100–133, http://dx.doi.org/10.1016/j.autcon.2015.12.003.
[20] W. Terkaj, A. Šojić, Ontology-based representation of IFC EXPRESS rules: An enhancement of the ifcOWL ontology, Autom. Constr. 57 (2015) 188–201, http://dx.doi.org/10.1016/j.autcon.2015.04.010.
[21] L. Fangzheng, K. Carsten, F. Yunyi, BIM based two-dimensional floor plan simulation and planning for evacuation, in: Forum Bauinformatik 2019, 2019, pp. 1–8.
[22] X. Zhou, J. Wang, M. Guo, Z. Gao, Cross-platform online visualization system for open BIM based on WebGL, Multimedia Tools Appl. 78 (2019) 1–14, http://dx.doi.org/10.1007/s11042-018-5820-0.
[23] P. Pauwels, A. Roxin, SimpleBIM: from full ifcOWL graphs to simplified building graphs, in: Proceedings of the 11th European Conference on Product and Process Modelling, ECPPM 2016, CRC Press, 2016, pp. 11–18.
[24] P. Pauwels, T. Krijnen, W. Terkaj, J. Beetz, Enhancing the ifcOWL ontology with an alternative representation for geometric data, Autom. Constr. 80 (2017) 77–94, http://dx.doi.org/10.1016/j.autcon.2017.03.001.
[25] B. Ouyang, Z. Wang, R. Sacks, Semantic enrichment of object associations across federated bim semantic graphs in a common data environment, in: European Conference on Product and Process Modeling 2022, 2022, pp. 1–8.
[26] D. Tomaszuk, D. Hyland-Wood, RDF 1.1: Knowledge representation and data integration language for the web, Symmetry 12 (1) (2020) http://dx.doi.org/10.3390/sym12010084.
[27] J.D. Fernández, M.A. Martínez-Prieto, RDF serialization and archival, in: S. Sakr, A. Zomaya (Eds.), Encyclopedia of Big Data Technologies, Springer, 2018, pp. 1–11, http://dx.doi.org/10.1007/978-3-319-63962-8_286-1.
[28] J. Pérez, M. Arenas, C. Gutiérrez, The semantics and complexity of SPARQL, in: Proceedings of the 5th International Semantic Web Conference, ISWC 2006, in: Lecture Notes in Computer Science, vol. 4273, Springer, 2006, pp. 30–43.
[29] W3C, RDF schema 1.1, 2014, http://www.w3.org/TR/rdf-schema/.
[30] T. Berners-Lee, J. Hendler, O. Lassila, The semantic web, Sci. Am. 284 (2001) 34–43, https://www.jstor.org/stable/26059207.
[31] W3C OWL Working Group, OWL 2 web ontology language profiles, 2012, https://www.w3.org/TR/owl2-profiles/.
[32] F. Baader, I. Horrocks, C. Lutz, U. Sattler, An Introduction To Description Logic, Cambridge University Press, 2017.
[33] S. Staab, R. Studer, Handbook on Ontologies, Springer-Verlag, 2009.
[34] A. Khamparia, B. Pandey, Comprehensive analysis of semantic web reasoners and tools: a survey, Educ. Inf. Technol. 22 (2017) 3121–3145, http://dx.doi.org/10.1007/s10639-017-9574-5.
[35] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, Z. Wang, HermiT: An OWL 2 reasoner, J. Automat. Reason. 53 (3) (2014) 245–269, http://dx.doi.org/10.1007/s10817-014-9305-1.
[36] E. Thomas, J. Pan, Y. Ren, TrOWL: tractable OWL 2 reasoning infrastructure, in: Proceedings of the 7th Extended Semantic Web Conference (ESWC 2010), Part II, in: Lecture Notes in Computer Science, vol. 6089, Springer, 2010, pp. 431–435, http://dx.doi.org/10.1007/978-3-642-13489-0_38.
[37] E. Sirin, B. Parsia, B.C. Grau, A. Kalyanpur, Y. Katz, Pellet: A practical OWL-DL reasoner, J. Web Semant. 5 (2) (2007) 51–53, http://dx.doi.org/10.1016/j.websem.2007.03.004.
[38] M. Horridge, S. Bechhofer, The OWL API: A java API for OWL ontologies, Semant. Web 2 (1) (2011) 11–21, http://dx.doi.org/10.3233/SW-2011-0025.
[39] A. Hogan, M. Arenas, A. Mallea, A. Polleres, Everything you always wanted to know about blank nodes, J. Web Semant. 27–28 (2014) 42–69, http://dx.doi.org/10.1016/j.websem.2014.06.004.
[40] N.M. Hoang, S. Törmä, Implementation and Experiments with an IFC-to-Linked Data Converter, in: Proceedings of the 32nd W78 Conference Applications of IT in the Architecture, Engineering and Construction Industry, 2015, pp. 285–294.
[41] M. Bonduel, J. Oraskari, P. Pauwels, M. Vergauwen, R. Klein, The IFC to linked building data converter: Current status, in: Proceedings of the 6th Linked Data in Architecture and Construction Workshop, LDAC 2018, in: CEUR Workshop Proceedings, vol. 2159, CEUR-WS.org, 2018, pp. 34–43, http://ceur-ws.org/Vol-2159/04paper.pdf.
[42] M. Rasmussen, M. Lefrançois, G. Schneider, P. Pauwels, BOT: the building topology ontology of the W3C linked building data group, Semant. Web 12 (1) (2020) 143–161, http://dx.doi.org/10.3233/SW-200385.
[43] D. Mavrokapnidis, K. Katsigarakis, P. Pauwels, E. Petrova, I. Korolija, D. Rovas, A Linked-Data Paradigm for the Integration of Static and Dynamic Building Data in Digital Twins, in: Proceedings of the 8th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, BuildSys 2021, ACM, 2021, pp. 369–372, http://dx.doi.org/10.1145/3486611.3491125.
[44] B. Balaji, A. Bhattacharya, G. Fierro, J. Gao, J. Gluck, D. Hong, A. Johansen, J. Koh, J. Ploennigs, Y. Agarwal, M. Bergés, D. Culler, R.K. Gupta, M.B. Kjærgaard, M. Srivastava, K. Whitehouse, Brick: Metadata schema for portable smart building applications, Appl. Energy 226 (2018) 1273–1292, http://dx.doi.org/10.1016/j.apenergy.2018.02.091.
[45] A.L. Richard Fellows, Research Methods for Construction, fourth ed., Wiley, 2015.
[46] M.A. Musen, The Protégé project: a look back and a look forward, AI Matters 1 (4) (2015) 4–12, http://dx.doi.org/10.1145/2757001.2757003.
[47] J.M. Keil, M. Gänß inger, The problem with XSD binary floating point datatypes in RDF, in: Proceedings of the 19th Extended Semantic Web Conference, ESWC, in: Lecture Notes in Computer Science, vol. 13261, Springer, 2022, pp. 165–182, http://dx.doi.org/10.1007/978-3-031-06981-9_10.
[48] I. Horrocks, O. Kutz, U. Sattler, The even more irresistible $\mathcal{SROIQ}$, in: Proceedings of the 10th International Conference of Knowledge Representation and Reasoning, KR 2006, 2006, pp. 57–67.
[49] D. Elshani, A. Lombardi, A. Fisher, S. Staab, D. Hernández, T. Wortmann, Knowledge graphs for multidisciplinary co-design: Introducing RDF to BHoM, in: Proceedings of the 10th Linked Data in Architecture and Construction Workshop, LDAC 2022, 2022, pp. 1–11.